

실제 데이터를 텐서로 표현해보기

4장에서 다루는 내용

- 파이토치 텐서로 실제 데이터 표현하기
- 데이터 타입 범위 다루기
- 파일에서 데이터 읽기
- 데이터를 텐서로 변환하기
- 신경망 모델 입력으로 사용하기 위한 텐서 만들기

이전 3장에서 우리는 텐서가 파이토치 데이터의 빌딩 블록임을 배웠다. 신경망은 텐서를 입력 받아 출력할 텐서를 만든다. 사실 신경망에서 일어나는 모든 연산과 최적화는 텐서 연산이며, 가중치나 바이어스 같은 신경망 내의 모든 파라미터도 텐서다. 텐서로 어떻게 연산을 수행하며 효과적으로 색인하는지 이해하는 것은 파이토치를 성공적으로 다루기 위한 핵심이다. 지금까지 텐서에 대한 기본 내용을 배웠으니 이제 실력을 더 키워보자.

앞에서도 질문했던 내용인데, 어떻게 하면 비디오나 텍스트 같은 데이터를 텐서로 표현해 딥러닝 모델을 훈련시키기에 적합하도록 가공할 수 있을까? 이 질문에 대한 해답이 바로 이번 4장에서 다룰 내용이다. 이번 장에서는 여러 다양한 종류의 데이터를 보면서 이 책에서 원하는 형태는 어떤 것이고 그렇게 만들기 위해 텐서로 표현하는 방법은 무엇인지 살펴보려 한다. 가장 널리 사용되는 일반 포맷으로 디스크에 저장된 데이터를 어떻게 읽어오는지 배우고, 읽은 데이터를 신경망 훈련을 위해 변환하는 관점에서 바라보게 될 것이다. 앞으로 실무에 적용하다 보면 원본 데이터가 문제 해결에 딱 들어맞지는 않는 경우를 자주 보게 될 것이다. 몇 가지 텐서 연산들을 좀더 배우며 이를 위한 텐서 데이터 조작을 익혀보자.

앞으로는 꽤 많은 이미지가 들어있거나 용적이 큰 데이터를 사용하게 된다. 실제 데이터가 대부분 이렇기도 하고 책 형태로 만들어내기도 용이하다. 관심있는 독자를 위해 테이블 형식의 데이터나 시계열, 그리고 텍스트 데이터도 다룰 예정이다. 백문이 불여일견이라고, 이미지 데이터부터 바로 시작하자. 그리고 나서 환자의 해부도를 표현한 큰 용적의 의료 데이터로 3차원 배열을 다루는 법을 알아보자. 그리고 와인 관련 테이블 데이터를 스프레드시트처럼 다뤄보자. 이후 공유 자전거 프로그램에 관한 시계열 데이터, 즉 순서가 있는 `ordered` 테이블 데이터를 살펴볼 것이다. 마지막으로 제인 오스틴 *Jane Austen*의 텍스트 데이터에 살짝 발을 담궈 볼 예정이다. 텍스트 데이터들은 순서를 유지하지만, 단어들을 숫자의 배열로 표현하는 것에 어떤 문제가 있는지도 알게 될 것이다.

대부분의 딥러닝 연구자가 그러하듯 설명을 이어가다가 데이터를 모델에 넣기 직전에 잠시 멈출 테니 이 시점에 데이터셋을 보관할 것을 추천한다. 그러면 그다음 장에서 신경망 모델을 어떻게 훈련시킬지를 배울 때 매우 유용하게 사용할 수 있다.

4.1 이미지 다루기

컨볼루션 신경망은 컴퓨터 비전에 혁명을 가져왔고(<http://mng.bz/zjMa>), 그때부터 이미지 기반 시스템에는 새로운 차원의 가능성이 열렸다. 각 부분을 심도 있게 튜닝한 알고리즘을 연결한 파이프라인에서 발생하던 이전의 문제들은 입력에 대응하는 출력 쌍으로 이뤄진 예제들로 신경망의 처음부터 끝까지 지나치며 훈련시키는 방식을 통해 유래 없는 수준의 성능으로 해결됐다. 이 놀라운 혁명에 동참해 보려면 우리가 쉽게 접할 수 있는 이미지 포맷을 읽어서 파이토치에서 기 대하는 방식에 맞춰 이미지의 다양한 부분을 담은 텐서 표현으로 데이터를 변환해야 한다.

이미지는 픽셀 단위의 높이와 너비를 가지는 표준적인 그리드에 나열된 복수 개의 스칼라 값 모음으로 표현된다. 그리드의 포인트, 즉 한 픽셀에 대한 하나의 스칼라 값을 가지는 흑백 이미지 혹은 앞서 3장에서 살펴본 것처럼 여러 색을 표현하는 여러 개의 스칼라 값을 가진다. 색상 외에 추가적으로 카메라에서 얻은 심도 등의 다양한 피처를 포함하기도 한다.

일반 카메라처럼 각 픽셀의 스칼라 값은 8비트 정수 인코딩을 사용한다. 의학이나 과학 혹은 군사용 애플리케이션에서는 12비트나 16비트처럼 이보다 높은 정밀도를 사용하는 경우를 자주 볼 수 있다. 더 높은 정밀도를 사용하면 뼈의 밀도나 온도, 심도 같은 물리적인 속성 정보를 픽셀에 인코딩할 때 더 넓은 범위나 높은 민감도를 가지도록 만들 수 있다.

4.1.1 컬러 채널 더하기

이전에 컬러에 대해 잠깐 설명했는데 컬러를 숫자로 인코딩하는 방법은 다양하다.¹ 제일 흔한 방법은 RGB로서, 빨강, 초록, 파랑의 강도를 나타내는 세 개 숫자로 컬러를 정의하는 방식이다. 각 컬러 채널은 흑백 이미지에서 픽셀 단위로 흑백의 강도를 나타내는 값을 가지는 맵으로 볼 수 있으며, 붉은 컬러에 대한 이미지의 맵을 보는 것은 마치 붉은색 선글라스를 끼고 그림을 보는 경우와 비슷하다고 생각하면 된다. 그림 4.1과 같은 무지개 그림의 경우, RGB의 각 채널은 전체 스펙트럼의 일부 영역에 대한 정보를 캡처한 셈이다(사실 주황색이나 노란색 같은 경우, 빨강과 초록을 섞어서 표현하게 되는데 그림에서는 이런 부분을 생략했다).

무지개의 붉은색 대역은 빨간색 채널에서 제일 밝게 표시되고, 같은 방식으로 무지개의 파랑과 하늘색이 파란색 채널에서 제일 밝게 나온다. 흰색 구름은 모든 채널에서 강도가 높게 나오는 것도 주목하자.

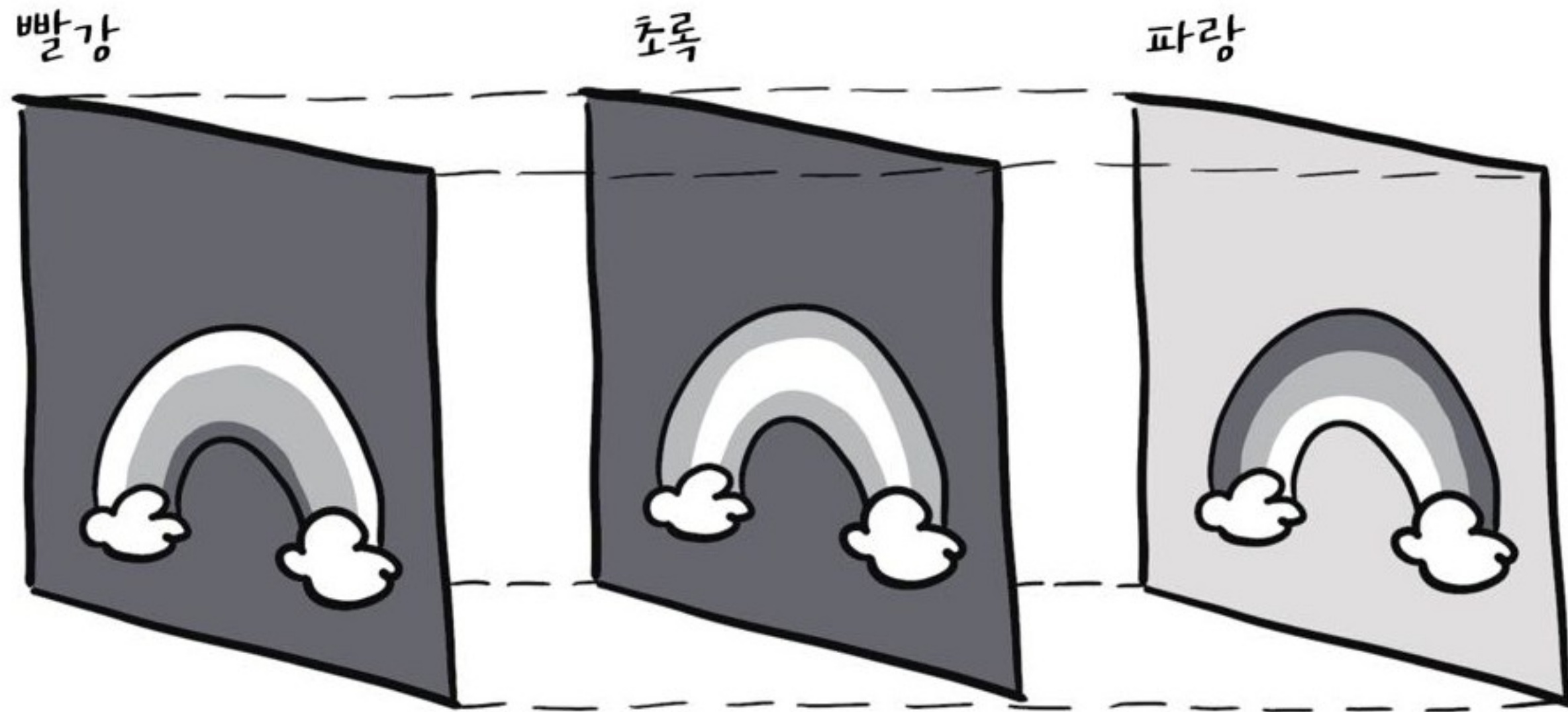


그림 4.1 빨강, 초록, 파랑 채널로 분리된 무지개

4.1.2 이미지 파일 로딩

이미지는 여러 파일 포맷으로 저장될 수 있지만 다행히도 파이썬에서는 다양한 방식으로 이미지를 로딩할 수 있다. `imageio` 모듈을 사용한 PNG 이미지 로딩부터 해보자([code/p1ch4/1_image_dog.ipynb](#)).

리스트 4.1 `code/p1ch4/1_image_dog.ipynb`

```
# In[2]:
import imageio

img_arr = imageio.imread('../data/p1ch4/image-dog/bobby.jpg')
img_arr.shape

# Out[2]:
(720, 1280, 3)
```

노트

4장에서는 단일화된 API로 다양한 데이터 타입을 다룰 수 있는 `imageio`를 사용한다. 토치비전(TorchVision)은 이미지와 비디오를 다루기 편리한 다목적 도구다. 여기서는 `imageio`를 사용해서 간단하게 처리해본다.

코드의 `img`는 너비와 높이에 해당하는 두 개의 공간 정보와 빨강, 초록, 파랑 채널에 해당하는 세 번째 차원까지 합하면 3차원 배열 같은 넘파이 객체다. 넘파이 배열을 리턴하는 라이브러리라면 어떤 것이든지 이로부터 파이토치 텐서를 얻을 수 있으며 각 차원의 레이아웃만 조심하면 된다. 이미지 데이터를 다루는 파이토치 모듈은 텐서가 $C \times H \times W$, 즉 채널, 높이, 너비 순으로 배치되어야 한다.

4.1.3 레이아웃 변경하기

구식의 차원 레이아웃에서 적절한 레이아웃을 얻기 위해 새로운 차원으로 변경하려면 텐서의 `permute` 메소드를 사용한다. 앞에서 얻은 $H \times W \times C$ 입력 텐서에 대해 채널2와 채널0 그리고 채널1 순으로 나열되도록 변경하면 적절한 레이아웃이 된다.

```
# In[3]:  
img = torch.from_numpy(img_arr)  
out = img.permute(2, 0, 1)
```

전에 본 적 있는 코드지만 이 연산은 텐서 복사본을 만들지 않는다는 점에 유의하자. `out`은 `img` 저장 공간과 동일하며 단순히 텐서 레벨에서 크기와 스트라이드 정보만 변경됐다. 비용이 매우 저렴한 연산이기 때문에 편리하다. 미리 힌트를 주자면, `img`에서 픽셀을 변경하는 경우 `out`에 있는 픽셀도 바뀐다는 점만 주의하자.

딥러닝 프레임워크마다 레이아웃이 다르다는 사실도 염두에 두자. 예를 들어 원래의 텐서플로는 채널 차원을 마지막에 배치하는 $H \times W \times C$ 레이아웃이었다(지금은 다양한 레이아웃을 지원한다). 저수준의 성능 관점에서 이 레이아웃은 장단점이 있지만 아무튼 텐서의 형태를 적절히 변경하는 한 차이는 없다.

지금까지는 하나의 이미지만 다뤘다. 앞서 사용했던 방식과 동일하게, 우리가 다룰 신경망의 입력으로 사용할 여러 장의 이미지 데이터셋을 만들어보자. 첫 번째 차원에 여러 이미지를 배치 `batch`로 넣어 $N \times C \times H \times W$ 텐서로 저장하자.

텐서를 만드는 작업을 좀더 효율적으로 하기 위해 `stack`을 사용해 미리 적당한 공간을 할당하고 디렉토리에서 읽은 이미지로 채워 넣을 수 있다.

```
# In[4]:  
batch_size = 3  
batch = torch.zeros(batch_size, 3, 256, 256, dtype=torch.uint8)
```

우리가 만들 배치의 높이는 256, 픽셀 너비는 256 픽셀이며, 이는 RGB 이미지 세 개로 이뤄진 배치임을 의미한다. 텐서 타입도 확인하자. 우리는 각 색상이 8비트 정수로 표현될 것을 기대하는데, 이게 바로 대다수 표준 카메라에서 다루는 사진 포맷이다. 이제 입력 디렉토리에서 PNG 이미지를 읽어 텐서에 저장할 수 있다.

```
# In[5]:
import os

data_dir = '../data/plch4/image-cats/'
filenames = [name for name in os.listdir(data_dir)
               if os.path.splitext(name)[-1] == '.png']
for i, filename in enumerate(filenames):
    img_arr = imageio.imread(os.path.join(data_dir, filename))
    img_t = torch.from_numpy(img_arr)
    img_t = img_t.permute(2, 0, 1)
    img_t = img_t[:3]
    batch[i] = img_t
```

투명도 알파 채널까지 있는 이미지도 있지만,
우리는 RGB만 사용하므로 첫 세 개 채널만 유지함

4.1.4 데이터 정규화

앞서 언급했듯이 신경망은 일반적으로 부동소수점 텐서를 입력으로 사용한다. 신경망은 입력값이 대략 0에서 1 사이거나 -1에서 1 사이일 때 훈련 성능이 가장 좋은 특징을 띤다(빌딩 블록이 정의되는 방식이 이런 특징에 영향을 미친다).

따라서 대부분의 경우 텐서를 부동소수점으로 캐스팅하고 픽셀값을 정규화한다. 부동소수점 캐스팅은 쉽지만 정규화는 조금 복잡하다. 정해놓은 입력 값의 범위를 통해 정규화된 값이 0과 1(혹은 -1와 1) 사이에 놓여야 하기 때문이다. 단순하게는, 픽셀 값을 부호 없는 8비트 정수의 최댓값인 255로 나눈다.

```
# In[6]:  
batch = batch.float()  
batch /= 255.0
```

다른 방법으로 입력 데이터의 평균과 표준 편차를 구해서 평균이 0이고 각 채널값이 표준 편차를 넘지 않게 만들기도 한다.

```
# In[7]:
n_channels = batch.shape[1]
for c in range(n_channels):
    mean = torch.mean(batch[:, c])
    std = torch.std(batch[:, c])
    batch[:, c] = (batch[:, c] - mean) / std
```

노트

전체 데이터셋을 다루는 방법은 아직 모르므로, 하나의 배치 이미지 묶음만 정규화했다. 실제로 이미지로 작업할 때는, 미리 모든 훈련 데이터에 대한 평균과 표준 편차를 계산해서 이렇게 사전에 계산되고 고정된 값으로 빼고 나누는 방법이 좋은 관례다. 2.1.4절의 이미지 분류기에 대한 전처리에서 다뤘던 내용이다.

입력에 대해서는 정규화 외에 기하학적인 변환으로, 회전이나 크기 변환, 자르기 같은 추가적인 작업을 할 수 있다. 이런 작업들이 훈련에 도움이 될 수도 있고, 이미지의 크기처럼 신경망이 요구하는 입력 형태를 위해 필요한 작업이 되기도 한다. 12.6절에서 이를 위한 몇가지 방법을 다루게 된다. 지금은 가능한 몇 가지 이미지 조작 옵션이 있다는 정도만 기억해두자.

4.2 3차원 이미지: 용적 데이터

지금까지 우리는 카메라로 찍은 사진과 같은 2차원 이미지를 어떻게 읽고 표시하는지를 배웠다. 의학 분야에서 사용하는 이미지 애플리케이션에는 컴퓨터 단층 촬영이라는 CT 스캔 데이터가 있는데, 통상 머리에서 발끝까지를 축으로 하는 연속된 단층 이미지가 있고 각 이미지는 신체 단면을 촬영한 것이다. CT 스캔은 밀도가 높아지는 차례대로 폐, 지방층, 수분, 근육, 뼈 같은 신체부위를 구별할 수 있는데 임상 장비의 화면상에서는 순서대로 어두운 쪽에서 밝은 쪽으로 매핑된다. 단층 사진의 각 포인트는 신체를 투과하여 검출기에 도달한 엑스레이의 양에 따라 밀도를 계산하는데, 센서에 들어온 값을 역합성 곱으로 계산해 전체 덩어리로 만들기 위한 복잡한 수식이 동원된다.

CT는 흑백 이미지처럼 하나의 밀도 채널만 있다. 즉 다른 색상 채널은 그대로 둔 채 데이터를 저장하기도 한다. 따라서 이전 절에서 살펴본 것처럼 통상 세 개 채널이 다 들어 있다. 그리고 이 2차원 단면을 스택처럼 쌓아 3차원 텐서를 만들면 대상의 3차원 해부도를 표현한 용적 데이터를 만들 수 있다. 그림 4.1에서 각 채널이 가시광선 스펙트럼의 특정 대역을 나타냈던 것과 달리, 그림 4.2의 추가 차원은 물리적인 오프셋 정보²를 나타낸다.

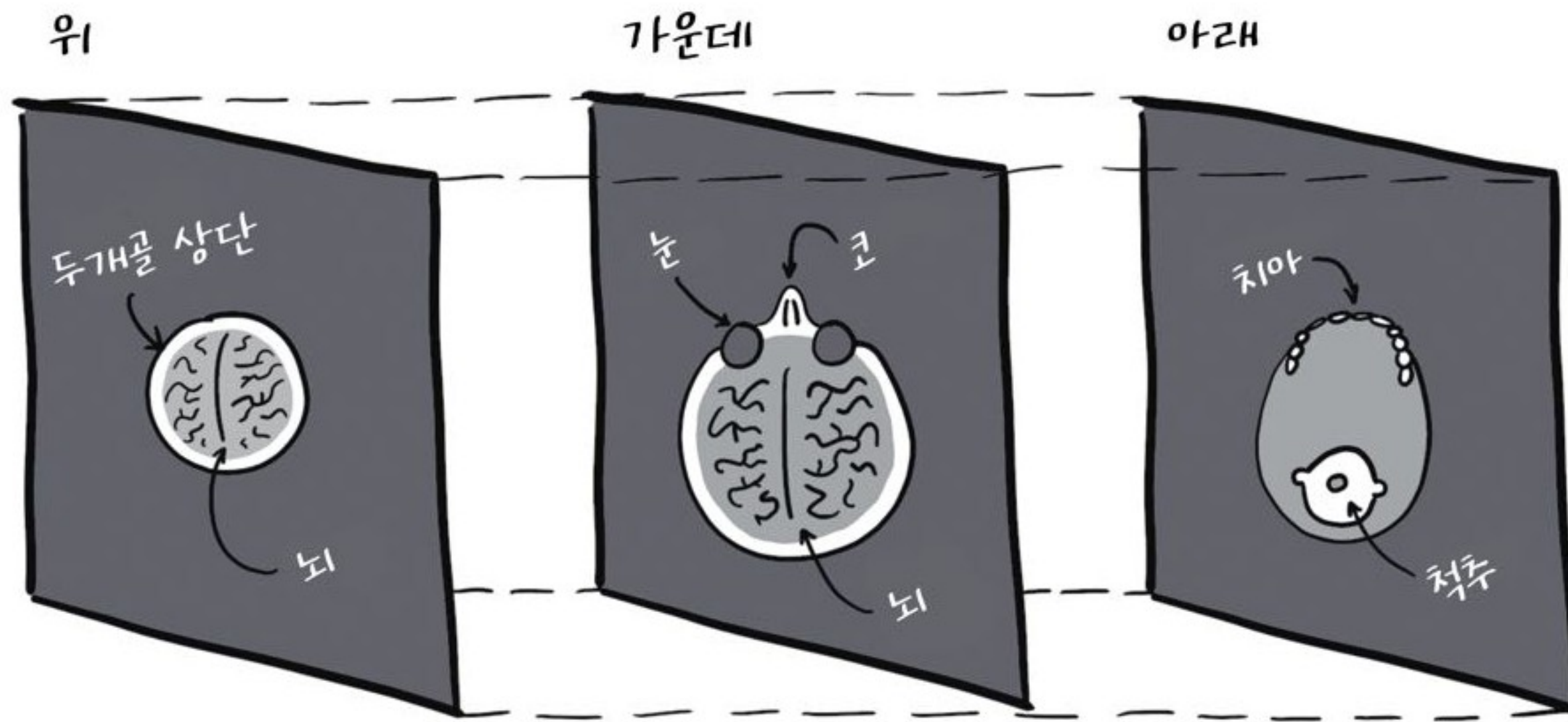


그림 4.2 정수리부터 턱선까지 나타내는 CT 스캔 단면

이 책의 2부에서 실제 의학 이미지 관련 문제를 상세하게 다루므로, 여기서 의학 이미지 포맷을 자세히 다루지는 않겠다. 지금은 그냥, 용적 데이터를 저장하는 텐서와 이미지 데이터는 근본적으로 큰 차이가 없다는 정도만 알아두자. 채널 차원 뒤에 부가적으로 깊이^{depth} 차원을 가지기 때문에 $N \times C \times D \times H \times W$ 의 5차원 텐서가 되는 것만 이해하자.

4.2.1 특수 포맷 로딩

샘플 CT 스캔 로딩에는 `imageio` 모듈에 있는 `volread` 함수를 사용한다. 이 함수는 인자로 받은 디렉토리에 있는 의료용 디지털 영상 `DICOM`, Digital Imaging and Communications in Medicine 파일³을 연속된 형태로 조합해서 3차원 넘파이 배열을 만든다(`code/plch4/2_volumetric_ct.ipynb`).

리스트 4.2 `code/plch4/2_volumetric_ct.ipynb`

```
# In[2]:
import imageio

dir_path = "../data/plch4/volumetric-dicom/2-LUNG 3.0 B70f-04083"
vol_arr = imageio.volread(dir_path, 'DICOM')
vol_arr.shape

# Out[2]:
Reading DICOM (examining files): 1/99 files (1.0%99/99 files (100.0%)
    Found 1 correct series.
Reading DICOM (loading data): 31/99 (31.392/99 (92.999/99 (100.0%)

(99, 512, 512)
```


4.1.3절에서는 정상이었지만 채널 정보가 사라져서 파이토치에서 예상하는 레이아웃과 달라졌다. `unsqueeze`를 사용해서 channel 차원을 위한 공간을 만들어보자.

```
# In[3]:  
vol = torch.from_numpy(vol_arr).float()  
vol = torch.unsqueeze(vol, 0)  
  
vol.shape  
  
# Out[3]:  
torch.Size([1, 99, 512, 512])
```

앞 절에서 했던 것처럼 batch 방향을 따라 여러 용적 데이터를 쌓으면 5차원 데이터셋을 만들 수 있다. 2부에서는 더 많은 CT 데이터를 살펴볼 것이다.

4.3 테이블 데이터 표현하기

머신러닝 작업을 할 때 접하게 되는 가장 단순한 형태의 데이터는 스프레드시트나 CSV 파일 혹은 데이터베이스에 들어 있는 경우다. 어디에 들어 있든지 간에, 샘플 혹은 레코드별로 한 행에 해당하는 데이터이고 그 안에 있는 각 열은 샘플에 대한 정보 한 조각을 포함하는 식이다.

여기에서 일단 테이블에 샘플이 들어 있는 순서는 아무 의미가 없다고 가정해보자. 샘플 안에 시간 순으로 정렬된 시계열 데이터와 달리, 각 샘플이 독립적으로 들어 있다고 생각해보자.

각 열은 특정 위치에서의 온도 같은 숫자값이나 샘플의 속성을 나타내는 ‘파랑’과 같은 문자열 등의 레이블을 가질 수 있다. 따라서 테이블 형식의 데이터는 일반적으로 각 열의 타입이 동일하지 않다. 즉 열이 다르면 데이터 타입도 다르다. 예를 들어 어떤 열에서는 사과의 무게를, 다른 열에서는 사과의 색상을 인코딩해 레이블에 담을 수 있다.

반면 파이토치 텐서의 내부 값들은 형태가 동일하다. 파이토치는 통상 숫자로 인코딩하며 정수나 부울 값을 가질 수도 있지만 대부분 부동소수점 값을 가진다. 당연히 그렇게 되는 이유는 신경망 자체가 실수값을 입력으로 받아 연속적인 행렬 곱셈이나 비선형 함수 같은 연속적인 연산을 거치며 실수 값을 출력으로 만들기 때문이다.

4.3.1 실세계 데이터셋 사용하기

딥러닝 실습을 위한 첫 번째 단계는 상이한 형태의 값으로 이뤄진 실세계 데이터를 부동소수점 수로 이뤄진 텐서로 만들어서 신경망으로 넣을 준비를 하는 것이다. 이미 인터넷상에 대량의 테이블 데이터셋이 많다. 예를 들어 <https://github.com/caesar0301/awesome-public-datasets> 같은 곳에서 재미있는 데이터를 찾아보자. 여기서 와인 데이터를 고르자! 와인 품질 데이터셋은 포르투갈 북부의 비뉴 베르데 *vinho verde* 와인 샘플의 화학적 특징이 기록된 데이터셋으로, 맛에 대한 점수도 포함되어 있다. 화이트 와인 데이터는 <http://mng.bz/90OI>에서 받을 수 있다. 편의상 이 책의 깃허브 사이트의 `data/p1ch4/tabular-wine` 디렉토리에도 복사본을 넣어뒀다.

파일을 열면, 각 열의 이름을 적은 헤더 줄이 제일 처음에 있고 이후에는 12개의 콤마로 구분된 값이 나열되어 있다. 처음 11개 열은 화학적 성분에 대한 값이고 마지막은 맛을 점수로 나타내어 0이 최저, 10이 최고의 맛을 뜻한다. 등장하는 순서대로 각 열의 이름을 나열하면 다음과 같다.

fixed acidity
volatile acidity
citric acid
residual sugar
chlorides
free sulfur dioxide
total sulfur dioxide
density
pH
sulphates
alcohol
quality

이 데이터셋에 대해 실행할 머신러닝 작업은 화학적 성분만 보고 맛을 예측하는 것이다. 머신러닝 결과가 뛰어나다고 해서 와인 맛을 평가하는 직업이 사라지는 것은 아니니 걱정은 금물이다. 먼저 훈련 데이터를 얻어야 한다. 이를 통해 그림 4.3에서처럼 화학적 성분 열과 품질 열과의 상관관계를 찾을 수 있다면 좋겠다. 예를 들어 여기서는, 황 성분이 줄어들수록 맛이 좋아지는 관계를 기대한다.

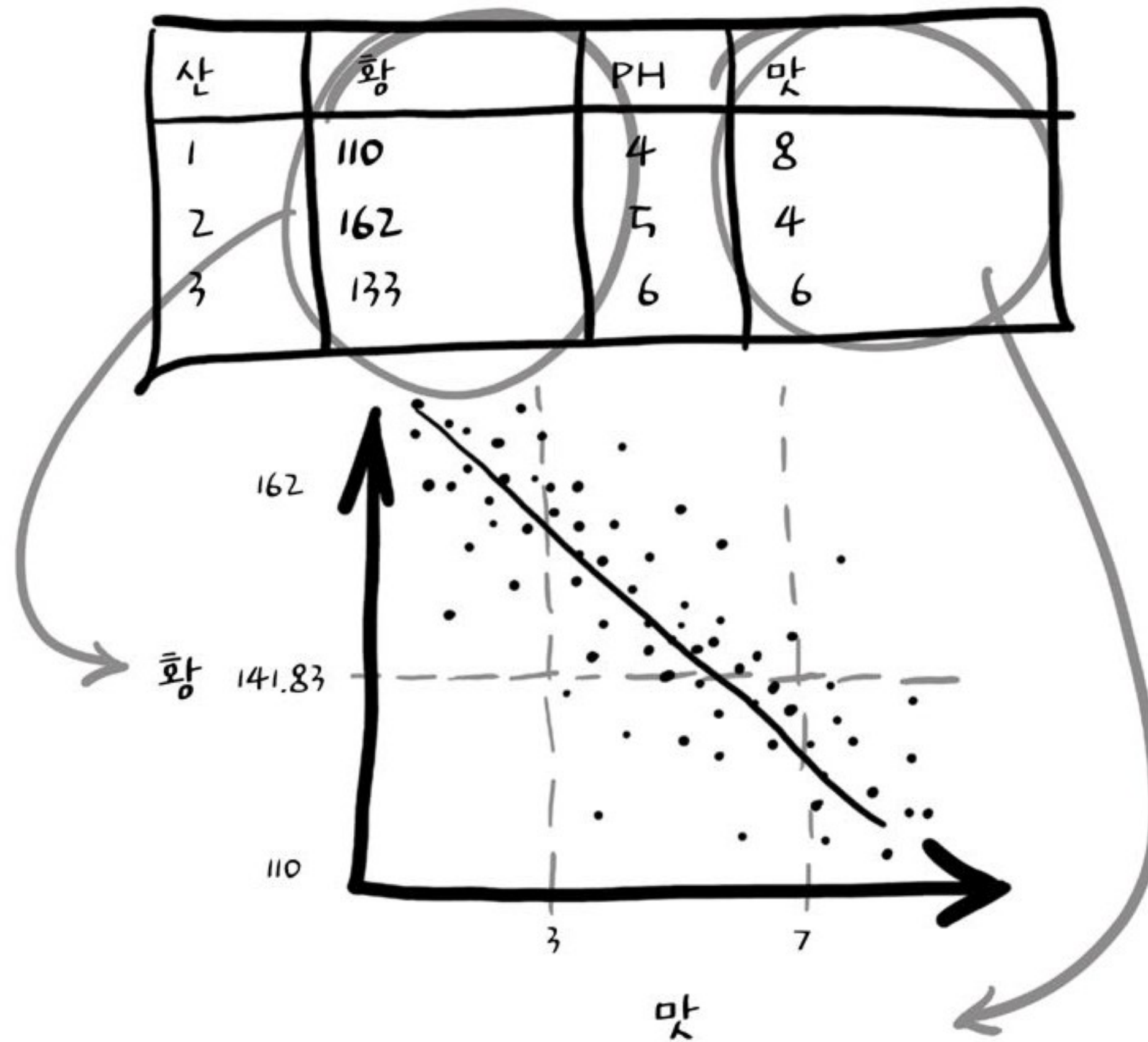


그림 4.3 (우리가 바라는) 와인에서의 황 성분과 맛과의 상관관계

4.3.2 와인 데이터를 텐서로 읽어오기

본격적인 분석에 들어가기에 앞서, 텍스트 편집기에서 파일을 여는 것보다도 더 편한 방식으로, 데이터를 들여다볼 수 있게 해야겠다. 파이썬으로 데이터를 읽은 후 어떻게 파이토치 텐서로 바꾸는지를 살펴보자. 파이썬에서는 여러 방법으로 CSV 파일을 간편하게 읽을 수 있다. 제일 흔한 세 가지 방법은 다음과 같다.

- 파이썬에 내장된 csv 모듈을 사용하는 법
- 넘파이NumPy
- 판다스Pandas

세 번째 판다스를 사용하는 방법이 가장 빠르고 메모리도 덜 쓴다. 하지만 파일 로딩 목적으로만 추가적인 라이브러리를 배우는 것은 좀 과하니, 이번에는 건너뛰자. 이미 앞 절에서 다룬 바 있기도 하고 파이토치와 넘파이는 호환도 잘 되므로, 넘파이를 사용하자. 이제 파일을 읽고, 만들어진 넘파이 배열을 파이토치 텐서로 바꾸자([code/p1ch4/3_tabular_wine.ipynb](#)).

리스트 4.3 code/p1ch4/3_tabular_wine.ipynb

```
# In[2]:
import csv
wine_path = "../data/p1ch4/tabular-wine/winequality-white.csv"
wineq_numpy = np.loadtxt(wine_path, dtype=np.float32, delimiter=";",
                        skiprows=1)

wineq_numpy

# Out[2]:
array([[ 7.  , 0.27, 0.36, ..., 0.45,  8.8 , 6. ],
       [ 6.3 , 0.3 , 0.34, ..., 0.49,  9.5 , 6. ],
       [ 8.1 , 0.28, 0.4 , ..., 0.44, 10.1 , 6. ],
       ...,
       [ 6.5 , 0.24, 0.19, ..., 0.46,  9.4 , 6. ],
       [ 5.5 , 0.29, 0.3 , ..., 0.38, 12.8 , 7. ],
       [ 6.  , 0.21, 0.38, ..., 0.32, 11.8 , 6. ]], dtype=float32)
```

만들어질 2차원 배열의 타입이 32비트 부동소수점이고 각 행을 구별하는 구분자도 정해주었으며 각 열의 이름을 나타내는 첫 행은 생략하도록 지정했다. 읽어온 데이터를 확인해보자.

```
# In[3]:  
col_list = next(csv.reader(open(wine_path), delimiter=';'))
```

```
wineq_numpy.shape, col_list
```

```
# Out[3]:  
((4898, 12),  
 ['fixed acidity',  
  'volatile acidity',  
  'citric acid',  
  'residual sugar',  
  'chlorides',  
  'free sulfur dioxide',  
  'total sulfur dioxide',  
  'density',  
  'pH',  
  'sulphates',  
  'alcohol',  
  'quality'])
```


그리고 넘파이 배열을 파이토치 텐서로 변환하자.

```
# In[4]:  
wineq = torch.from_numpy(wineq_numpy)  
  
wineq.shape, wineq.dtype  
  
# Out[4]:  
(torch.Size([4898, 12]), torch.float32)
```

이제 맛 점수가 기록된 마지막 열까지 들어간 부동소수점 `torch.Tensor`를 만들었다.

사용할 데이터를 배워보기 전에 먼저 세 가지 종류의 숫자값을 살펴보자.⁴ 첫 번째는 **연속값**(continuous value)으로, 숫자로 표현했을 때 가장 직관적인 값이기도 하다. 값 사이의 순서가 정해져 있으며 두 값의 차이가 의미를 지닌다. A라는 패키지가 B보다 2킬로그램 더 무겁다고 치자. 혹은 B 패키지는 A보다 100마일 먼 곳에서 왔다고 가정해보자. 이렇게 표현하면 A가 3킬로그램이든 10킬로그램이든, 혹은 B가 200마일 떨어진 곳에서 왔든지 2,000마일 떨어진 곳에서 왔든지 상관없이 원래의 의미에 영향을 주지 않는다. 특정 단위를 사용해 어떤 값을 계속적으로 세거나 측정한다면 이 연속값에 해당할 가능성이 크다. 글로 서술하는 방법으로 이와 같은 연속값을 좀더 잘게 쪼갤 수 있다. 말하자면 앞의 예에서 어떤 것이 두 배 무겁다라거나 세 배 더 멀다라는 표현은 비율을 사용해서 값을 기술하는 셈이다. 반면 시각을 나타낼 때는 이런 식으로 하지 않는다. 즉 6시가 3시보다 두 배 느리다는 식으로 말하지 않는다. 하루 중의 시각은 일정한 주기 단위로 표현하기 때문이다.

다음으로는 **순서값**(ordinal value)이 있다. 연속값에서처럼 값은 연속적으로 나열되어 있지만 값 사이의 관계가 고정되어 있지 않다. 좋은 예로 음료수를 대, 중, 소와 같이 용량 순으로 나열하는 경우를 들 수 있다. 제일 작은 것부터 차례로 나눠 소(小) 사이즈를 1, 중(中)은 2, 대(大)는 3으로 매핑할 수 있다. 3이 2보다 크듯이 대자 음료수는 중자 음료수보다 크다. 하지만 얼마만큼 큰지는 알 수가 없다. 음료수 크기를 나타내는 세 개의 숫자 1, 2, 3을 실제 크기인 각각 8온스, 12온스, 24온스로 두면 간격이 있는 값으로 변환하는 셈이다. 순서값은 순서를 다룰 때 사용해야지, 3과 1의 평균값을 구해서 중자 음료수 크기를 구하는 식의 계산을 해서는 안 된다!

마지막인 **카테고리값**(categorical value)은 순서나 숫자적인 의미가 없는 값이다. 가능한 모든 경우를 나열하기 위해 임의의 수를 사용한다고 생각하면 된다. 물은 1, 커피는 2, 탄산음료는 3, 우유는 4라고 부여하는 식이다. 물을 먼저 놓고 우유를 나중에 놓아야 하는 이유는 없다. 단순히 구별하기 위해 숫자를 부여하는 게 전부다. 커피에 10을, 그리고 우유에 -3을 지정해도 아무 차이가 없다(실제로 이런 경우에는 값을 부여할 때 0에서 N-1 사이의 값을 부여하면, 의미상의 차이는 없지만 4.5.4절에서 다룰 원핫 인코딩(one-hot encoding)과 임베딩(embedding)이 편리하긴 하다). 숫자값은 아무 의미도 부여되어 있지 않기 때문에 명목 척도(nominal scale)라고도 한다.

4.3.3 점수 표현하기

품질 점수를 연속값으로 취급해서 실수 형태를 유지한 채로 회귀 작업을 수행할 수 있고 레이블로 취급해서 분류 작업에서 화학 정보를 분석해 레이블을 예측하는 시도를 할 수도 있다. 통상 두 경우 모두, 입력 데이터의 텐서에서 점수를 제거해 독자적인 텐서로 유지하고, 모델에 입력하는 대신 점수를 정답 값으로 사용할 수 있다.

```
# In[5]:
data = wineq[:, :-1]  ← 모든 행과 마지막을 제외한 모든 열을 선택
data, data.shape

# Out[5]:
(tensor([[ 7.00, 0.27, ..., 0.45,  8.80],
        [ 6.30, 0.30, ..., 0.49,  9.50],
        ...,
        [ 5.50, 0.29, ..., 0.38, 12.80],
        [ 6.00, 0.21, ..., 0.32, 11.80]]), torch.Size([4898, 11]))

# In[6]:
target = wineq[:, -1] ← 모든 행과 마지막 열을 선택
target, target.shape

# Out[6]:
(tensor([6., 6., ..., 7., 6.]), torch.Size([4898]))
```


레이블 텐서의 target 텐서를 전치할 수 있는 두 가지 방법이 있는데, 카테고리 데이터를 어떤 용도로 쓰느냐에 따라 다르다. 한 가지 방법은 다음과 같이 점수를 담은 정수 벡터로 레이블을 처리하는 것이다.

```
# In[7]:  
target = wineq[:, -1].long()  
target
```

```
# Out[7]:  
tensor([6, 6, ..., 7, 6])
```

‘와인 색’처럼 값이 문자열로 이루어진 레이블이라면 각 문자열마다 대응하는 정수를 할당해서 같은 식으로 처리할 수 있다.

4.3.4 원핫 인코딩

점수를 처리하는 또 다른 방법으로 원핫 인코딩(one-hot encoding)이 있다. 1부터 10까지의 값이 벡터 안의 10개의 원소에 대응하도록 정해두고서 원소 하나만 1로 설정하고 나머지는 모두 0으로 설정하는 방법이다. 예를 들어 값 1은 벡터 (1, 0, 0, 0, 0, 0, 0, 0, 0, 0)으로 나타내고 5는 (0, 0, 0, 0, 1, 0, 0, 0, 0, 0)으로 나타내는 식이다. 점수를 인코딩하면서 공교롭게도 1이 나타난 순서가 점수와 일치했는데 분류 관점에서 보면 다른 순서로 정해도 상관없다.

두 가지 방식에는 차이점이 있다. 와인의 품질 점수를 정수 벡터로 처리하는 것은 점수상에 순서가 있음을 가정하며 이 와인의 예에서는 적절하다. 품질 점수 1이 4보다는 낮은 것이 당연한 것처럼 말이다. 그뿐만 아니라 이 경우에는 점수 간의 거리도 가정한다. 점수 1과 3의 차이는 점수 2와 4의 차이와 동일하다고 가정하게 된다. 실제 품질도 점수 관계와 동일하다면 괜찮다. 만일 점수가 포도 품종처럼 서로 간에 완전히 이산적인 경우라면 값 사이의 순서나 거리 개념이 없는 원핫 인코딩이 더 낫다. 원핫 인코딩은 2.4처럼 정수 점수 사이의 분수 값은 의미가 없으며 이것 아니면 저것처럼 양적인 점수를 따지는 경우에 적합하다. 원핫 인코딩은 `scatter_` 메소드를 사용한다. 이 메소드는 소스 텐서와 함께 전달된 인자의 인덱스를 따라 새 텐서를 채워 넘겨준다.


```
# In[8]:
target_onehot = torch.zeros(target.shape[0], 10)

target_onehot.scatter_(1, target.unsqueeze(1), 1.0)

# Out[8]:
tensor([[0., 0., ..., 0., 0.],
        [0., 0., ..., 0., 0.],
        ...,
        [0., 0., ..., 0., 0.],
        [0., 0., ..., 0., 0.]])
```

`scatter_` 메소드는 어떤 역할을 할까? 일단 이름의 마지막에 `_`이 붙어 있다. 3장에서 살펴본 `바`와 같이, 파이토치에서 이름 끝에 `_`이 붙으면 새로운 텐서를 반환하는 메소드가 아니라, 텐서를 바꿔치기 하는 방법으로 변경하는 메소드임을 알 수 있다. `scatter_`의 인자는 다음과 같다.

- 뒤에 오는 두 개의 인자가 따라야 하는 차원을 명세함
- 원핫으로 인코딩할 요소를 가리키는 인덱스가 들어있는 텐서
- 원핫 인코딩할 원소가 들어있는 텐서 혹은 단일 스칼라(이 경우에는 1)

이 호출문에서는 “각 행에서 대상이 되는 레이블의 인덱스를 취한 다음(이 경우에는 와인 품질 점수와 일치한다) 열 인덱스 값으로 사용해서 해당 항목의 값을 1.0으로 지정한다”는 의미다. 그 결과, 카테고리 정보를 인코딩한 텐서가 만들어진다.

scatter_의 두 번째 인자인 인덱스 텐서는 텐서를 원할 인코딩할 때 동일한 차원 수를 만들기 위해 필요하다. target_onehot이 2차원($4,898 \times 10$)이므로 unsqueeze를 사용해 target에 추가 차원을 만들어준다.

```
# In[9]:
target_unsqueezed = target.unsqueeze(1)
target_unsqueezed
```

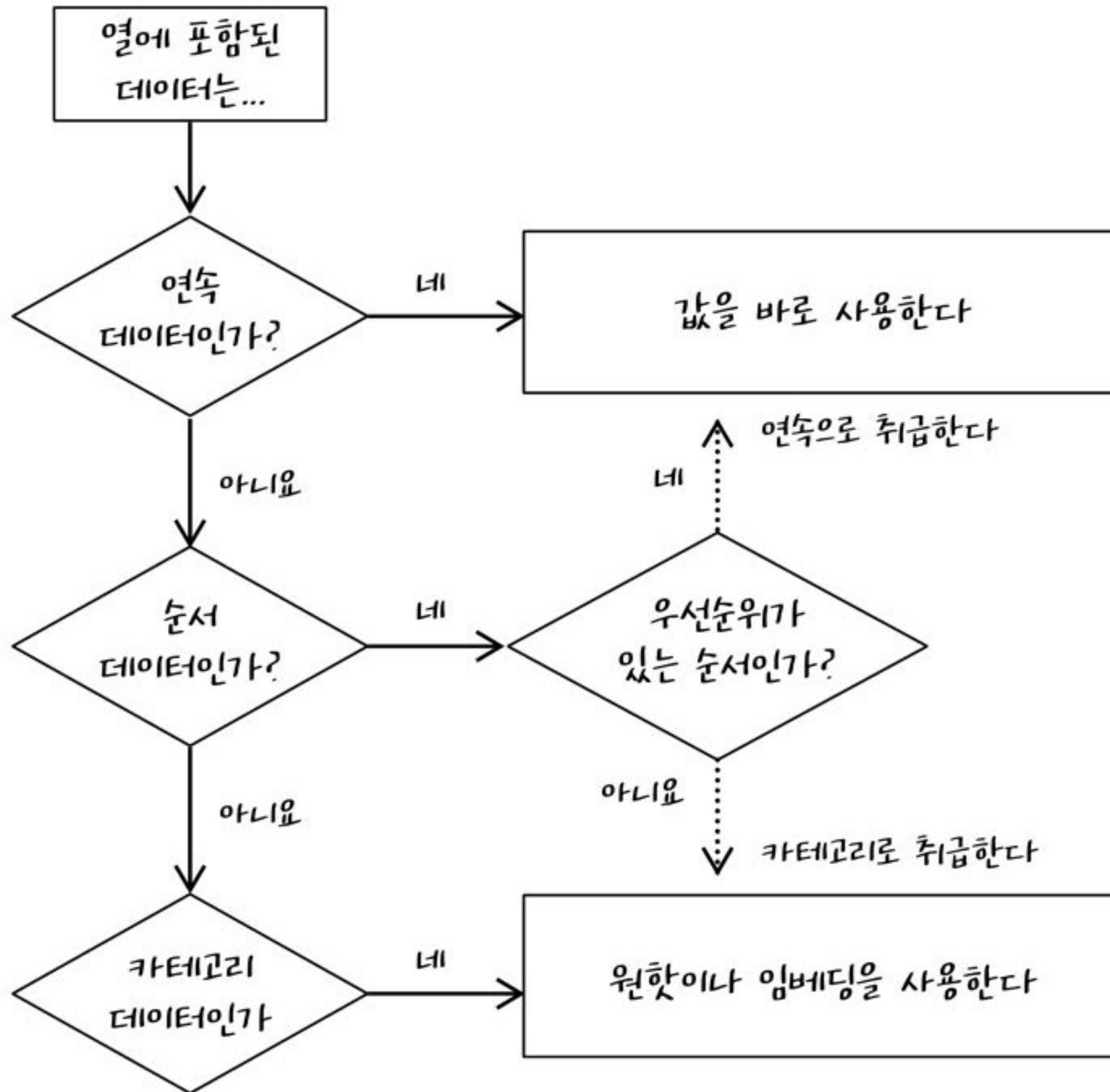
```
# Out[9]:
tensor([[6],
        [6],
        ...,
        [7],
        [6]])
```

`unsqueeze`는 **싱글톤** `singleton` 차원을 추가한다. 4,898개의 요소가 있는 1차원 텐서가 값의 변화나 추가 요소 없이 $(4,898 \times 1)$ 크기의 2차원 텐서로 바뀐다. 요소를 접근하기 위한 인덱스가 추가로 만들어졌다고 생각하면 된다. 이전에는 `target`의 첫 요소를 `target[0]`로 접근했다면, 동일한 요소를 `unsqueezed`인 텐서에서는 `target_unsqueezed[0, 0]`로 접근한다.

파이토치는 신경망 훈련 과정에서 클래스 인덱스를 직접 사용할 수 있게 해준다. 그렇다고 해도 신경망 입력으로 들어가는 점수를 카테고리 입력으로 쓰고 싶다면 이를 원핫 인코딩된 텐서로 바꿔야 한다.

4.3.5 언제 카테고리화 할 것인가

연속 데이터와 카테고리 데이터 다루는 법을 둘 다 살펴보았다. 아마 앞서 살펴본 순서 데이터까지 비롯해 이 세 가지로 데이터를 어떻게 다뤄야 할지 궁금할 것이다. 사실 여기에 정해진 답은 없다. 우리의 데이터는 (순서 정보를 포기하고, 카테고리가 몇 개 안 될 경우에는 훈련 과정에서 카테고리를 찾아내길 바라며) 카테고리 데이터로 취급하거나 (임의로 값 사이의 거리를 전제하며) 연속 데이터로 취급할 수 있다. 연속값으로 다루는 경우는 나중에 그림 4.5에 있는 날씨 데이터를 예로 들어 설명할 예정이다. 그림 4.4에 간단한 순서도로 데이터 매핑을 요약했다.



실제값 사례

3.1415

0 0 0 0 1 0 0 0 0

그림 4.4 연속 데이터, 순서 데이터, 카테고리 데이터로 열을 다루는 방법

화학 분석값이 들어 있는 11개의 변수가 포함된 data 텐서로 돌아가보자. 파이토치 텐서 API의 함수로 텐서 형태의 데이터를 가공해보겠다. 먼저 각 열의 평균과 표준 편차를 구하자.

```
# In[10]:
```

```
data_mean = torch.mean(data, dim=0)
```

```
data_mean
```

```
# Out[10]:
```

```
tensor([6.85e+00, 2.78e-01, 3.34e-01, 6.39e+00, 4.58e-02, 3.53e+01,  
        1.38e+02, 9.94e-01, 3.19e+00, 4.90e-01, 1.05e+01])
```

```
# In[11]:
```

```
data_var = torch.var(data, dim=0)
```

```
data_var
```

```
# Out[11]:
```

```
tensor([7.12e-01, 1.02e-02, 1.46e-02, 2.57e+01, 4.77e-04, 2.89e+02,  
        1.81e+03, 8.95e-06, 2.28e-02, 1.30e-02, 1.51e+00])
```


이 경우 `dim=0`은 차원 0에 대해 축소^{reduction} 연산⁵이 수행됨을 지정한다. 이제 학습 과정에 도움이 되도록 열의 값에서 평균을 빼고 표준편차로 나눠 정규화를 진행할 수 있다. 정규화는 다음 5장의 5.4.4절에서 자세히 다룬다.

```
# In[12]:  
data_normalized = (data - data_mean) / torch.sqrt(data_var)  
data_normalized
```

```
# Out[12]:  
tensor([[ 1.72e-01, -8.18e-02, ..., -3.49e-01, -1.39e+00],  
        [-6.57e-01, 2.16e-01, ..., 1.35e-03, -8.24e-01],  
        ...,  
        [-1.61e+00, 1.17e-01, ..., -9.63e-01, 1.86e+00],  
        [-1.01e+00, -6.77e-01, ..., -1.49e+00, 1.04e+00]])
```

4.3.6 임계값으로 찾기

대충 눈으로 데이터를 직접 들여다보고 좋은 와인과 나쁜 와인을 구별할 쉬운 방법이 있는지 살펴보자. 먼저 점수가 3 이하인 열을 target에서 걸러보자.

```
# In[13]:  
bad_indexes = target <= 3  
bad_indexes.shape, bad_indexes.dtype, bad_indexes.sum()  
  
# Out[13]:  
(torch.Size([4898]), torch.bool, tensor(20))
```

← 파이토치의 비교 함수인 torch.le(target, 3)도 가능하지만
이와 같이 연산자를 사용하는 방식이 더 표준적임

True로 설정된 `bad_indexes`는 20개뿐이다! 고급 인덱싱^{advanced indexing}이라 부르는 파이토치 기능이 있으므로 `data` 텐서를 인덱싱할 때 `torch.bool` 데이터 타입을 사용하면 열이 True에 해당하는 행들만 접근할 수 있다. `bad_indexes` 텐서의 차원 정보는 `target`과 동일하며 원래의 `target` 텐서에 들어 있는 요소에 대해 임계값으로 비교하여 False와 True에 따라 골라낸 상태다.

```
# In[14]:  
bad_data = data[bad_indexes]  
bad_data.shape
```

```
# Out[14]:  
torch.Size([20, 11])
```


새 `bad_data` 텐서에는 20개 행이 있는데, 이는 `bad_indexes` 텐서에서 `True` 값을 가진 행의 수와 동일하다. 다른 11개의 열은 그대로다. 이런 식으로 이제 와인 정보를 좋음, 보통, 나쁨 카테고리로 나눌 수 있다. 각 열에서 `.mean()` 값을 뽑아보자.

```
# In[15]:
```

```
bad_data = data[target <= 3]
```

```
mid_data = data[(target > 3) & (target < 7)]
```

```
good_data = data[target >= 7]
```

← & 연산자를 통한 부울 넘파이 배열과
파이토치 텐서의 논리 and 연산

```
bad_mean = torch.mean(bad_data, dim=0)
```

```
mid_mean = torch.mean(mid_data, dim=0)
```

```
good_mean = torch.mean(good_data, dim=0)
```

```
for i, args in enumerate(zip(col_list, bad_mean, mid_mean, good_mean)):
```

```
    print('{:2} {:20} {:6.2f} {:6.2f} {:6.2f}'.format(i, *args))
```

```
# out[15]:
```

0	fixed acidity	7.60	6.89	6.73
1	volatile acidity	0.33	0.28	0.27
2	citric acid	0.34	0.34	0.33
3	residual sugar	6.39	6.71	5.26
4	chlorides	0.05	0.05	0.04
5	free sulfur dioxide	53.33	35.42	34.55
6	total sulfur dioxide	170.60	141.83	125.25
7	density	0.99	0.99	0.99
8	pH	3.19	3.18	3.22
9	sulphates	0.47	0.49	0.50
10	alcohol	10.34	10.26	11.42

눈으로 확인할 수 있는 특징이 보이기 시작했다. 한눈에 봐도, 일단 나쁜 와인은 이산화황^{sulfur dioxide} 성분이 다른 경우보다 높은 듯하다. 와인의 좋고 나쁨을 판단하기 위한 대강의 평가 기준으로 이산화황 총량^{total sulfur dioxide}을 임계값으로 사용할 수 있을 것 같다. 이산화황 총량에서 앞서 계산해 놓은 중앙점보다 낮은 인덱스만 가져와보자.

```
# In[16]:
total_sulfur_threshold = 141.83
total_sulfur_data = data[:,6]
predicted_indexes = torch.lt(total_sulfur_data, total_sulfur_threshold)

predicted_indexes.shape, predicted_indexes.dtype, predicted_indexes.sum()

# Out[16]:
(torch.Size([4898]), torch.bool, tensor(2727))
```

이 임계값으로는 전체 와인 중 절반 이상을 높은 품질로 구별할 수 있을 것 같다. 이제 실제로 좋은 와인의 인덱스를 뽑아보자.

```
# In[17]:
```

```
actual_indexes = target > 5
```

```
actual_indexes.shape, actual_indexes.dtype, actual_indexes.sum()
```

```
# Out[17]:
```

```
(torch.Size([4898]), torch.bool, tensor(3258))
```


임계값으로 예측한 수량보다 좋은 와인은 500개나 더 많이 존재하므로 완벽하지 않다는 구체적인 증거를 이미 확보한 셈이다. 실제 순위와 우리가 예측한 순서가 얼마나 잘 맞는지 확인할 필요가 있다. 예측한 순위와 실제 품질 순위에 논리 and 연산을 수행해서 얻은 교집합으로 평가해보자(각각은 0과 1의 배열임을 기억하자). 그리고 이를 사용해서 얼마나 잘 했는지 판단하자.

```
# In[18]:
n_matches = torch.sum(actual_indexes & predicted_indexes).item()
n_predicted = torch.sum(predicted_indexes).item()
n_actual = torch.sum(actual_indexes).item()

n_matches, n_matches / n_predicted, n_matches / n_actual

# Out[18]:
(2018, 0.74000733406674, 0.6193984039287906)
```

대략 2,000여 개의 와인을 맞췄다! 2,700개를 예측했으니 74%의 확률로 고품질의 와인을 걸러냈다. 하지만 안타깝게도 실제 좋은 와인의 수는 3,200개이고 이 중 61%만 구별해냈다. 단순한 방법으로 구별했으니 그에 맞는 결과를 얻었다고 생각하면 된다. 랜덤으로 예측한 것보다 조금 나은 수준이다! 당연한 결과다. 실제로는 와인의 품질에 당연히 더 많은 변수들이 작용하고 이 변수들과 (이진 숫자가 아닌 실제 점수로 나오는) 품질 사이에는 숫자 하나를 경계값으로 사용하는 것보다 훨씬 복잡한 관계가 존재한다.

여기에 간단한 신경망을 적용하면 다른 기본적인 머신러닝 방법처럼 한계를 넘어설 수 있다. 이어지는 5장과 6장을 읽으며 여러분의 첫 신경망을 바닥부터 만들고 익히면 이 문제를 해결하기 위한 도구로 활용할 수 있을 것이다. 더 나은 평가를 위해 어떻게 해야할지에 대해서는 12장에서 다시 다루겠다. 이제 다른 형태의 데이터 타입을 다뤄보자.

4.4 시계열 데이터 다루기

지금까지 앞 절에서는 테이블로 구조화된 데이터를 어떻게 표현하는지를 다뤘다. 테이블의 모든 행은 다른 행과 독립적이었으며 행의 순서는 중요하지 않았다. 어떤 행이 앞에 있고 또 뒤에 있는지에 대한 정보를 인코딩한 열은 없었다.

와인 데이터셋으로 다시 돌아가서 생각해보자. 만일 매년 와인의 품질이 어떻게 발전해왔는지를 알 수 있는 year 열이 있었다면 어떨까? 안타깝게도 그런 데이터는 아직 없지만 이 책의 개정판에 넣으려고 각 와인 병마다 데이터 샘플을 수작업으로 모으는 중이다. 그동안 다른 재미있는 데이터셋을 살펴보자. 여기 워싱턴 DC로부터 얻은 2011년부터 2012년 사이의 캐피탈 바이크쉐어Capital BikesShare 공유 자전거 시스템의 시간대별 자전거 대여 데이터에, 날씨와 계절 정보까지 포함된 데이터셋을 <http://mng.bz/jgOx>에서 구할 수 있다. 2차원의 데이터셋을 그림 4.5처럼 3차원 데이터셋으로 바꿔보자.

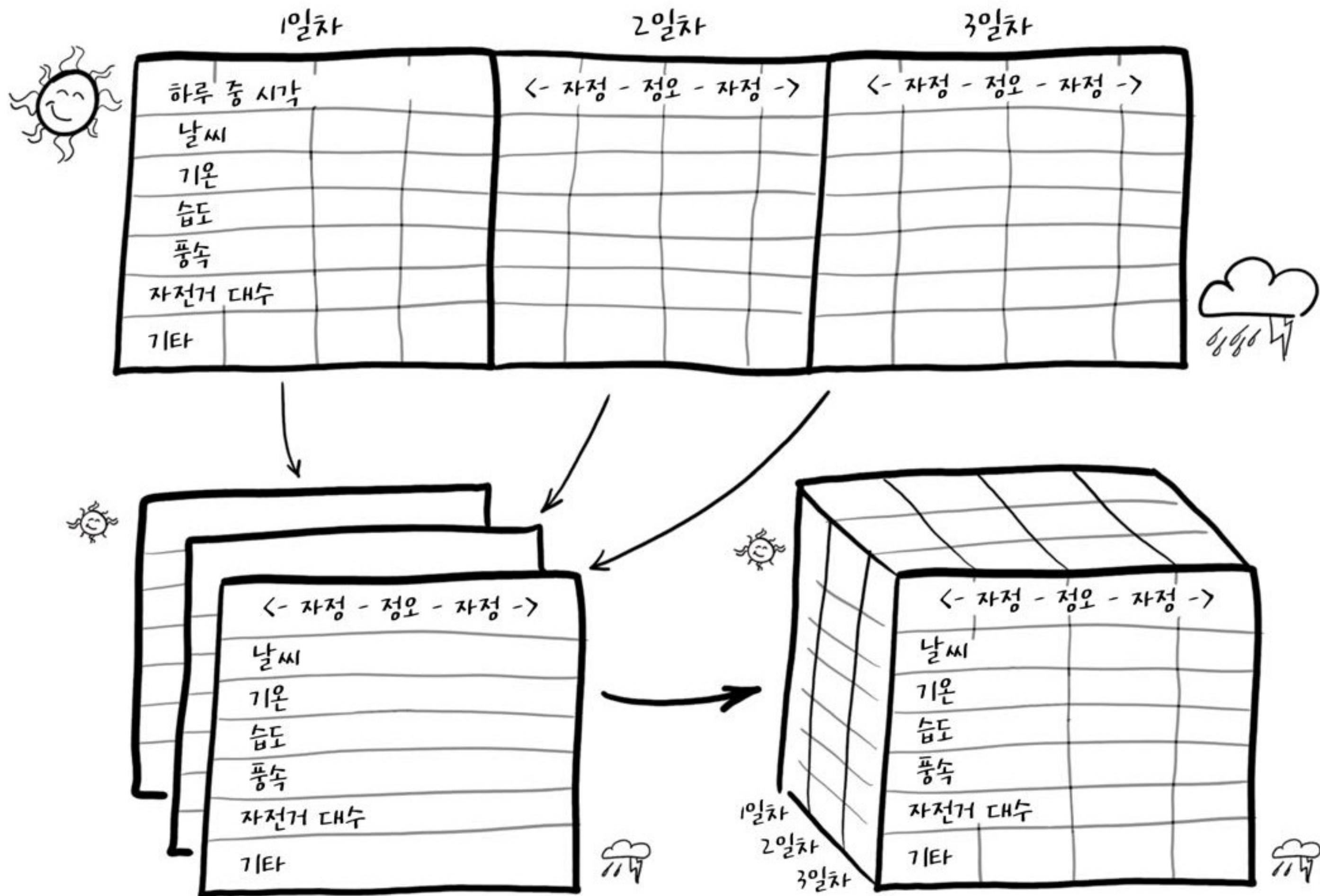


그림 4.5 1차원 다채널 데이터셋의 각 샘플을 일자와 시각을 별도 축으로 하여 2차원 다채널 데이터셋으로 변환했다

4.4.1 시간 차원 더하기

소스 데이터에서 각 행은 시간별로 분리된 데이터다(그림 4.5에서는 책의 지면 관계상 행과 열을 바꿔 그렸다). 시간 행으로 되어 있는 구성을 바꿔 인덱스를 올릴 때마다 일자가 바뀌고 또 다른 축에서는 (날짜와 상관없이) 하루의 시간을 표현하게 만들자. 나머지 축에는 데이터의 각 열에 해당하는 날씨와 기온 등의 데이터가 포함된다.

이제 데이터를 읽어들이자(`code/p1ch4/4_time_series_bikes.ipynb`).

리스트 4.4 code/p1ch4/4_time_series_bikes.ipynb

```
# In[2]:
bikes_numpy = np.loadtxt(
    "../data/p1ch4/bike-sharing-dataset/hour-fixed.csv",
    dtype=np.float32,
    delimiter=",",
    skiprows=1,
    converters={1: lambda x: float(x[8:10])}) ← 첫 번째 열의 일자 문자열을 숫자로 변환
bikes = torch.from_numpy(bikes_numpy)
bikes

# Out[2]:
tensor([[1.0000e+00, 1.0000e+00, ..., 1.3000e+01, 1.6000e+01],
        [2.0000e+00, 1.0000e+00, ..., 3.2000e+01, 4.0000e+01],
        ...,
        [1.7378e+04, 3.1000e+01, ..., 4.8000e+01, 6.1000e+01],
        [1.7379e+04, 3.1000e+01, ..., 3.7000e+01, 4.9000e+01]])
```

매 시간대별 데이터셋의 정보는 다음과 같다.

- 레코드 인덱스: instant
- 일자: day
- 계절: season (1: 봄, 2: 여름, 3: 가을, 4: 겨울)
- 연도: yr (0: 2011, 1: 2012)
- 월: mnth (1에서 12)
- 시간: hr (0에서 23)
- 공휴일 상태: holiday
- 요일: weekday
- 평일 상태: workingday
- 날씨 상태: weathersit (1: 맑음, 2: 안개, 3: 약한 비/눈, 4: 강한 비/눈)
- 섭씨 기온: temp
- 체감 섭씨 기온: atemp
- 습도: hum
- 풍속: windspeed
- 일반 사용자: casual
- 등록 사용자: registered
- 대여 자전거 수: cnt

이와 같은 시계열 데이터셋에서 각 행은 연속적인 시간상에서의 한 지점을 표현한다. 각 행이 순서대로 나열되며 만들어진 차원이 존재한다. 각 행을 시간과 상관없는 것으로 여기고 이전 시간의 상황은 고려하지 않은 상태에서 단순히 몇 시에 몇 대의 자전거를 대여했는지 예측할 수도 있다. 하지만 순서가 있는 이 데이터는 시간에 따른 데이터 사이의 관계를 분석할 기회를 준다. 예를 들어 이전 시간대에 비가 오고 있었다면, 이 사실을 바탕으로 현재 시간대에서의 자전거 사용률을 예측할 수 있다. 일단은 먼저 신경망이 이 공유 자전거 데이터셋을 고정된 크기로 처리할 수 있도록 만드는 방법을 생각해보자.

이 신경망 모델은 대여 수나 하루 중 시각, 혹은 기온이나 날씨 상태 등의 각기 다른 값에 대해 연속으로 여러 개의 값을 볼 필요가 있다. 크기가 C 인 N 개의 병렬 시퀀스로 표현이 가능하다. 여기서 C 는 신경망 용어로 채널(channel)을 나타내는데, 여기서 볼 수 있는 데이터와 같이 1차원 데이터를 위한 열(column)과 동일하다. N 차원은 시간 축을 표현하며, 여기서는 한 차원당 한 시간을 의미한다.

4.4.2 시간 단위로 데이터 만들기

2년치 데이터셋을 쪼개서 일 단위처럼 다소 넓은 관찰 주기로 나눌 필요가 있을 지도 모르겠다. 이러면 길이가 L 인 C 개의 시퀀스를 가지는 N (샘플 수)개의 컬렉션을 얻게 된다. 달리 말하면 시계열 데이터셋이 3차원 텐서가 되어 $N \times C \times L$ 로 만들어진다. C 에는 17개의 채널이 들어가고, 1을 한 시간으로 해서 L 은 24가 된다. 하루 단위의 일반적인 리듬이 예측을 유리하게 만드는 패턴을 제공할 가능성이 높음에도 불구하고 24시간으로 나눠 사용하는 것에 특별한 이유는 없다. 원한다면 주 단위로 $7 \times 24 = 168$ 시간의 블록을 사용할 수도 있다. 어떻게 나누느냐는 행이 24나 168의 배수가 되기만 한다면 활용 가능하다. 그리고 당연히 시계열 사이에 빈틈을 둘 수는 없다.

자전거 공유 데이터셋으로 돌아가보자. 첫 열은 전역 레벨에서 데이터의 순서를 나타내는 인덱스이고 두 번째는 일자, 여섯 번째는 하루 중 시각 열이다. 일별 자전거 탑승 횟수와 필요한 외부 변수가 있으니 데이터셋을 만들 준비가 됐다. 데이터셋은 이미 정렬되어 있지만, 만일 아니라면 `torch.sort`로 적절하게 정렬하면 된다.

노트

우리가 사용하는 `hour-fixed.csv` 데이터셋은 원본 데이터셋에 없는 행을 보정한 데이터다. 없는 시간대의 행은 대부분 새벽 시간대인데 자전거 대여 수를 0으로 가정했다.

일별로 매 시간의 데이터셋을 구하기 위해 동일 텐서를 24시간 배치로 바라보는 뷰가 필요하다.

bikes 텐서의 차원 정보`shape`와 스트라이드`stride`를 확인하자.

```
# In[3]:  
bikes.shape, bikes.stride()  
  
# Out[3]:  
(torch.Size([17520, 17]), (17, 1))
```

17,520시간에 17개 열이다. 이제 데이터를 일자, 시간, 17개 열의 세 개 축으로 만들자.

```
# In[4]:  
daily_bikes = bikes.view(-1, 24, bikes.shape[1])  
daily_bikes.shape, daily_bikes.stride()  
  
# Out[4]:  
(torch.Size([730, 24, 17]), (408, 17, 1))
```

어떻게 됐을까? 먼저 `bikes.shape[1]`은 `bikes` 텐서의 열의 개수로 17이다. 그런데 여기에 중요한 건 사실 `view` 호출 부분이다. 텐서가 저장 공간에 있는 동일한 데이터를 다르게 바라보는 역할을 하기 때문이다.

앞 장에서 배웠듯이 텐서에 `view`를 호출하면 저장 공간을 바꾸지 않고 차원 수나 스트라이드 정보를 바꾼 새 텐서를 넘겨준다. 즉 어떤 데이터도 복사하지 않으므로 연산 비용을 들이지 않고도 텐서를 바꾼 것과 같은 효과를 낸다. `view` 호출에는 새로 받을 텐서가 가질 차원 정보를 넘겨줘야 한다. -1은 '남은 차원과 거기에 들어있는 요소를 다 합친 것'을 의미하는 플레이스홀더 `placeholder`로 사용한다.

역시 앞 장에 나온 내용이지만, 저장 공간은 연속된 숫자를 저장하는 컨테이너(이 경우에는 부동소수점)라는 점을 기억하자. `bikes` 텐서의 저장 공간에는 각 행이 순서대로 들어 있다. 이 사실은 예제에서 `bikes.stride()`를 호출해 확인했다.

`daily_bikes`에 대한 스트라이드를 보면 두 번째 차원인 시간을 한 칸 이동하려면 저장 공간상에서 열의 개수인 17칸만큼 이동해야 하고 첫 번째 차원인 일자를 한 칸 이동하려면 한 행의 길이에 24를 곱한 수, 즉 이 예에서는 17에 24를 곱한 408이 된다.

가장 오른쪽 차원은 원래 데이터셋에서와 같이 열의 수가 된다. 가운데 차원은 연속된 24시간을 나타내는 시간 차원이다. 표현을 달리하면 C 개의 채널을 가진 하루를 L 시간으로 나눈 N 개의 연속된 값을 가진다고 볼 때, 이를 $N \times C \times L$ 순서로 놓고 싶다면 텐서를 전치해야 한다.

```
# In[5]:
daily_bikes = daily_bikes.transpose(1, 2)
daily_bikes.shape, daily_bikes.stride()

# Out[5]:
(torch.Size([730, 17, 24]), (408, 1, 17))
```

이제 앞에서 배운 기술을 이 데이터셋에 적용해보자.

4.4.3 훈련 준비

‘날씨 상태’를 나타내는 데이터는 순서값이다. 총 4단계로 나뉘어 있는데, 1은 좋은 날씨고 4는 정말 안 좋은 날씨다. 이 값을 카테고리로 간주해 각 단계를 레이블로 볼 수 있고 동시에 연속값으로 볼 수도 있다. 카테고리로 보려면 값을 원핫 인코딩 벡터로 만들어 데이터셋과 병합해야 한다.⁶

데이터를 쉽게 렌더링하기 위해 먼저 첫 날의 데이터만 생각해보자. 하루 시간대 만큼의 행과 날씨 상태 가질수 만큼의 열을 가진, 0으로 채워진 행렬을 초기화한다.

```
# In[6]:
first_day = bikes[:24].long()
weather_onehot = torch.zeros(first_day.shape[0], 4)
first_day[:,9]

# Out[6]:
tensor([1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 2, 2,
        2, 2])
```

그리고 날씨 수준에 따라 행렬을 원핫 인코딩으로 변환하자. 이전 절에서 했던 것처럼 `unsqueeze`를 사용하여 싱글톤 차원을 더해준다.

```
# In[7]:
weather_onehot.scatter_(
    dim=1,
    index=first_day[:,9].unsqueeze(1).long() - 1, ← 날씨값은 1부터 4까지만,
    value=1.0)                                   색인은 0에서 시작하므로 1을 빼줌

# Out[7]:
tensor([[1., 0., 0., 0.],
        [1., 0., 0., 0.],
        ...,
        [0., 1., 0., 0.],
        [0., 1., 0., 0.]])
```

이 예에서 하루의 날씨는 1로 시작해서 2로 끝났다. 맞는 것 같다.

마지막으로 만들어진 행렬을 원래 데이터셋에 cat 함수를 사용해 병합한다. 결과에서 첫 부분을 살펴보자.

```
# In[8]:
torch.cat((bikes[:24], weather_onehot), 1)[:1]

# Out[8]:
tensor([[ 1.0000,  1.0000,  1.0000,  0.0000,  1.0000,  0.0000,  0.0000,
          6.0000,  0.0000,  1.0000,  0.2400,  0.2879,  0.8100,  0.0000,
          3.0000, 13.0000, 16.0000,  1.0000,  0.0000,  0.0000,  0.0000]])
```

이제 원래의 bikes 데이터셋에 성공적으로 ‘날씨 상태’를 원핫 인코딩하여 데이터 열이 있는 차원(1번 차원)을 따라 병합했다. 바꿔 말하면 두 개의 데이터셋 열이 겹겹이 쌓이게 됐다고 볼 수 있으며, 원래의 데이터셋 뒤에 새로운 원핫 인코딩 열이 붙었다고 생각할 수도 있다. cat으로 붙이기 위해서는 텐서에 붙을 차원의 크기가 같아야 한다. 이 예에서는 데이터 행의 길이에 해당한다. 새로운 마지막 네 개 열이 1, 0, 0, 0인 것을 보면, 기대했던 대로 날씨값 1로 끝나는 것을 확인할 수 있다.

`daily_bikes` 텐서에 대해서도 동일한 작업을 할 수 있다. 이 데이터는 (B, C, L) 형태이고 $L = 24$ 임을 기억하자. 일단 값이 0이고 크기가 각각 B 와 L , 그리고 추가 열의 크기는 C 인 텐서를 만들자.

```
# In[9]:  
daily_weather_onehot = torch.zeros(daily_bikes.shape[0], 4,  
                                   daily_bikes.shape[2])
```

```
daily_weather_onehot.shape
```

```
# Out[9]:  
torch.Size([730, 4, 24])
```

다음으로 텐서의 *C*차원에 원핫 인코딩을 넣어놓는다. 이 연산은 텐서 안에서 수행하므로 텐서의 내용이 바뀐다.

```
# In[10]:
daily_weather_onehot.scatter_(
    1, daily_bikes[:,9,:].long().unsqueeze(1) - 1, 1.0)
daily_weather_onehot.shape

# Out[10]:
torch.Size([730, 4, 24])
```


그리고 C차원을 따라 원핫 인코딩을 병합하자.

```
# In[11]:  
daily_bikes = torch.cat((daily_bikes, daily_weather_onehot), dim=1)
```

앞서 언급했듯이 이 방법이 '날씨 상태' 값을 처리하는 유일한 방법은 아니다. 레이블 자체에 순서 관계가 있으므로 연속 변수에 해당하는 특수값으로 가장할 수도 있다. 값을 변환해서 0.0을 1.0으로 바꿀 수도 있다.

```
# In[12]:  
daily_bikes[:, 9, :] = (daily_bikes[:, 9, :] - 1.0) / 3.0
```

마찬가지로 앞에서 다루던 것과 같이 변수를 [0.0, 1.0] 사이값으로 범위 변경을 하거나 [-1.0, 1.0] 사이값으로 변경하는 것은 모든 정량적인 변수를 다루는 방법인데, 현재 데이터셋의 10번째 열인 temperature가 이에 대응한다. 왜 그렇게 하는지는 나중에 다루기로 하고, 일단은 이 방식이 훈련 과정에 도움이 된다고만 알아두자.

값의 범위를 변경하는 방법은 다양하다. 첫 번째로, 다음과 같이 값의 범위를 [0.0, 1.0]으로 매핑할 수도 있다.

```
# In[13]:
temp = daily_bikes[:, 10, :]
temp_min = torch.min(temp)
temp_max = torch.max(temp)
daily_bikes[:, 10, :] = ((daily_bikes[:, 10, :] - temp_min)
                        / (temp_max - temp_min))
```

혹은 두 번째로, 다음과 같이 모든 값에서 평균을 빼고 표준편차로 나누기도 한다.

```
# In[14]:  
temp = daily_bikes[:, 10, :]  
daily_bikes[:, 10, :] = ((daily_bikes[:, 10, :] - torch.mean(temp))  
                        / torch.std(temp))
```

두 번째 방식에서는 평균이 0이고 단위 표준 편차를 가지는 값으로 바뀐다. 가우시안 분포 Gaussian distribution로 그리면, 샘플의 68%는 $[-1.0, 1.0]$ 에 분포하게 된다.

좋다! 쓸만한 데이터셋을 만들어봤고 시계열 데이터 다루는 법도 배웠다. 전체적으로 시계열 데이터의 배치 방식과 이러한 데이터를 신경망이 처리 가능하도록 만드는 아이디어를 살펴봤다.

시계열과 비슷한 형태의 종류로서 순서가 있는 데이터는 여럿 있는데, 그중 두 개를 뽑자면 텍스트와 오디오를 들 수 있다. 다음 절에서는 텍스트 데이터를 살펴보고 4.6 ‘결론’ 절에서 부가적으로 오디오 예제 링크를 둘러보자.

4.5 텍스트 표현하기

딥러닝은 자연어 처리^{natural language processing}(이하 NLP) 분야도 폭풍처럼 강타했다. 특별히 모델의 이전 출력과 현재의 입력을 섞는 식으로 반복해서 소비하는 형태의 모델이 대표적이다. 이런 모델을 **순환 신경망**^{RNN, recurrent neural network}라고 부르는데 텍스트 분류와 생성, 그리고 자동 번역 시스템에 적용해 뛰어난 성공을 거뒀다. 최근에는 **트랜스포머**^{transformer}로 불리는 종류의 신경망으로 과거의 정보를 포함하는 유연한 방법을 통해 큰 성공을 거두고 있다. 기존의 NLP는 언어의 문법을 인코딩한 규칙이 들어 있는 여러 스테이지를 가진 파이프라인으로 대표됐다.⁷ 최신 연구 결과는 이러한 규칙이 데이터로부터 훈련을 통해 유추되도록 다량의 말뭉치만 가지고 처음부터 끝까지 신경망을 훈련시킨다. 최근 수년 동안 인터넷상의 자동 번역 서비스는 대부분 딥러닝으로 동작한다.

이번 절의 목표는 텍스트를 이전처럼 신경망이 처리할 수 있는 숫자의 텐서로 바꾸는 것이다. 이렇게 할 수 있다면, 나중에 텍스트 처리 작업에 적절한 아키텍처를 고르고 파이토치로 NLP 작업을 수행할 수 있을 것이다. 이 모든 과정이 얼마나 강력한지는 즉시 확인할 수 있으며, 동일한 파이토치 도구로 도메인별 작업들에 대해 최첨단 기술에 육박하는 성능을 내도록 만들 수도 있다. 즉 일단 문제를 올바른 형태로 변환하기만 하면 된다. 이 작업의 첫 단계는 데이터의 차원 정보를 재정의하는 것이다.

4.5.1 텍스트를 숫자로 변환하기

신경망으로 텍스트를 다루는 직관적인 방법은 두 가지가 있다. 하나는 문자 단위로 한 번에 하나의 문자를 처리하는 것이고, 다른 하나는 단어 단위로 신경망이 바라보는 세밀한 엔티티로 개별 단어를 처리하는 것이다. 이때 어떤 방법이든지 텍스트 정보를 텐서로 인코딩하는 기술은 동일하다. 양쪽 방법 모두 신비로운 과정은 아니다. 앞에서 원핫 인코딩을 통해 이미 접한 바 있다.

문자 단위부터 보자. 먼저 처리할 텍스트를 구한다. 여러 사람이 자원해서 문학 작품을 디지털화하고 보관하는 구텐베르크 프로젝트(www.gutenberg.org)에는 일반 텍스트를 비롯해 무료로 사용 가능한 공개 형태의 굉장한 자료들이 많다. 만일 대용량의 말뭉치를 원한다면 위키피디아 말뭉치가 있는데, 190억 개 단어와 440만 개에 이르는 모든 위키피디아 문서를 모아놓은 것이다. 그 밖의 다른 말뭉치는 영어 말뭉치 웹사이트(www.english-corpora.org)에서 찾아볼 수 있다.

구텐베르크 웹사이트에서 제인 오스틴의 『오만과 편견』(www.gutenberg.org/files/1342/1342-0.txt)을 가져오자. 파일을 저장하고 `code/p1ch4/5_text_jane_austen.ipynb`에서 읽으면 된다.

리스트 4.5 `code/p1ch4/5_text_jane_austen.ipynb`

```
# In[2]:  
with open('../data/p1ch4/jane-austen/1342-0.txt', encoding='utf8') as f:  
    text = f.read()
```


4.5.2 문자 원형 인코딩

진행하기 전에 한 가지 주의할 구체적인 사항이 있다. 인코딩에 관한 것인데 어마어마한 주제라서 살짝 짚고 넘어가는 수준으로만 파악해보자. 모든 활자는 코드로 표현된다. 코드는 적당한 길이의 비트로 되어 있고 각 문자가 구별되어 인식되게 하는 역할을 한다. 가장 단순한 형태의 인코딩으로는 1960년대에 나온 ASCII(American Standard Code for Information Interchange)가 있다. ASCII는 128개의 문자를 128개의 정수로 인코딩한다. 예를 들어 a는 이진수 1100001이나 십진수 97과 동일하며, b는 이진수 1100010 혹은 십진수 98과 같은 식이다. 이 인코딩은 8비트로 충분히 표현 가능하며, 1965년에는 꽤 유용한 방식이었다.

노트

영어가 아닌 다른 언어에서 문장을 적절히 표현하기 위해 필요한 글리프(glyph), 강조 부호, 합자 등을 처리하기에 128개 문자만으로는 턱없이 부족하다. 그래서 더 넓은 범위의 문자를 다루도록 더 많은 비트를 사용하는 여러 가지 방식의 인코딩이 고안됐다. 넓은 범위의 문자 인코딩 기법은 유니코드(Unicode)라는 이름으로 표준화되어 알려진 모든 문자를 숫자로 매핑하게 됐고, 이 숫자를 비트로 표현하는 방식은 특정 인코딩 형식으로 제공됐다. 잘 알려진 인코딩으로는 UTF-8, UTF-16, UTF-32가 있는데, 각각 숫자를 8비트, 16비트, 32비트 정수로 표현한다. 파이썬 3.x에서 문자열은 유니코드를 따른다.

이제 우리는 문자를 원핫 인코딩할 예정이다. 여기에서 중요한 점은 분석 대상 텍스트에 있는 문자 집합으로 한정하여 원핫 인코딩하는 것이다. 예제의 경우 영어 텍스트를 로딩했으므로 ASCII를 사용한 작은 크기의 인코딩을 다루고 있기에 안전하다. 문자를 다 소문자로 만들어서 인코딩에서의 전체 문자 종류를 줄일 수도 있다. 비슷하게 구두점이나 숫자, 혹은 기대하지 않은 종류의 문자나 표기를 빼버릴 수도 있다. 이런 처리가 신경망의 결과에 실질적인 차이를 미칠지 여부는 작업의 종류에 따라 다르다.

그리고 나서 텍스트에서 문자를 파싱하고 각각에 대해 원핫 인코딩을 제공하면 된다. 인코딩에 들어있는 문자의 종류 수가 벡터의 크기가 되어 각 문자가 이 크기의 벡터로 표현된다. 벡터 내 각 요소는 인코딩에서 문자에 대응하는 위치를 제외하고 0으로 표현된다.

텍스트를 행으로 나누고 몇 줄을 가져와 보자.

```
# In[3]:
```

```
lines = text.split('\n')
```

```
line = lines[200]
```

```
line
```

```
# Out[3]:
```

```
'"Impossible, Mr. Bennet, impossible, when I am not acquainted with him'
```


행 전체의 문자를 원핫 인코딩한 문자의 총 수를 담은 텐서를 만들자.

```
# In[4]:
letter_t = torch.zeros(len(line), 128)  ← ASCII 제한인 128로 하드코딩함
letter_t.shape

# Out[4]:
torch.Size([70, 128])
```

이제 letter_t는 행마다 원핫 인코딩된 문자 하나를 담고 있다. 이제 각 행이 일치하는 문자를 표현하도록 정확한 위치에 1을 기록하자. 값이 1이어야 할 인덱스는 인코딩에서의 문자의 인덱스에 대응한다.

```
# In[5]:
for i, letter in enumerate(line.lower().strip()):
    letter_index = ord(letter) if ord(letter) < 128 else 0  ←
    letter_t[i][letter_index] = 1                          방향이 있는 쌍따옴표처럼 ASCII에
                                                            유효하지 않은 문자는 여기서 버림
```

4.5.3 모든 단어를 원핫 인코딩하기

신경망이 이해할 수 있는 표현으로 문장을 원핫 인코딩해봤다. 단어 단위 인코딩은 같은 식이지만 출현된 단어로 사전을 만들어 문장에 나오는 단어 시퀀스에 대해 한 단어를 한 행으로 원핫 인코딩한다. 사전에 포함되는 단어가 매우 많아 인코딩 벡터가 매우 길어지면 실용성이 떨어진다. 이를 위해 임베딩^{embedding}을 사용하여 단어 단위로 텍스트를 표현하는 효율적인 방법은 다음 절에서 살펴보고, 여기에서는 원핫 인코딩이 어떻게 전개되는지를 알아보자.

텍스트를 받아 소문자로 바꾸고 구두점을 날리는 `clean_words`를 정의하자. “Impossible, Mr. Bennet”가 포함된 line을 인수로 `clean_words`를 호출하면 다음과 같은 결과를 얻는다.

```

# In[6]:
def clean_words(input_str):
    punctuation = '.,;:"!?"_-'
    word_list = input_str.lower().replace('\n', ' ').split()
    word_list = [word.strip(punctuation) for word in word_list]
    return word_list

words_in_line = clean_words(line)
line, words_in_line

# Out[6]:
('"Impossible, Mr. Bennet, impossible, when I am not acquainted with him',
['impossible',
 'mr',
 'bennet',
 'impossible',
 'when',
 'i',
 'am',
 'not',
 'acquainted',
 'with',
 'him'])

```


다음으로 인코딩에서 단어를 인덱스로 매핑하자.

```
# In[7]:  
word_list = sorted(set(clean_words(text)))  
word2index_dict = {word: i for (i, word) in enumerate(word_list)}  
  
len(word2index_dict), word2index_dict['impossible']  
  
# Out[7]:  
(7261, 3394)
```

이제 word2index_dict는 단어를 키로, 정수를 값으로 가지는 사전^{dictionary}이다. 원핫 인코딩을 하면서 단어에 대한 인덱스를 효율적으로 찾을 용도로 사용할 것이다. 다음으로 문장을 처리할 단계다. 문장을 단어로 나누고 각각을 원핫 인코딩하면, 단어 하나당 원핫 인코딩된 벡터가 모여 텐서 하나를 만든다. 빈 벡터를 만들고 문장 내 각 단어의 원핫 인코딩 값을 부여하자.

```
# In[8]:
word_t = torch.zeros(len(words_in_line), len(word2index_dict))
for i, word in enumerate(words_in_line):
    word_index = word2index_dict[word]
    word_t[i][word_index] = 1
    print('{:2} {:4} {}'.format(i, word_index, word))

print(word_t.shape)
```

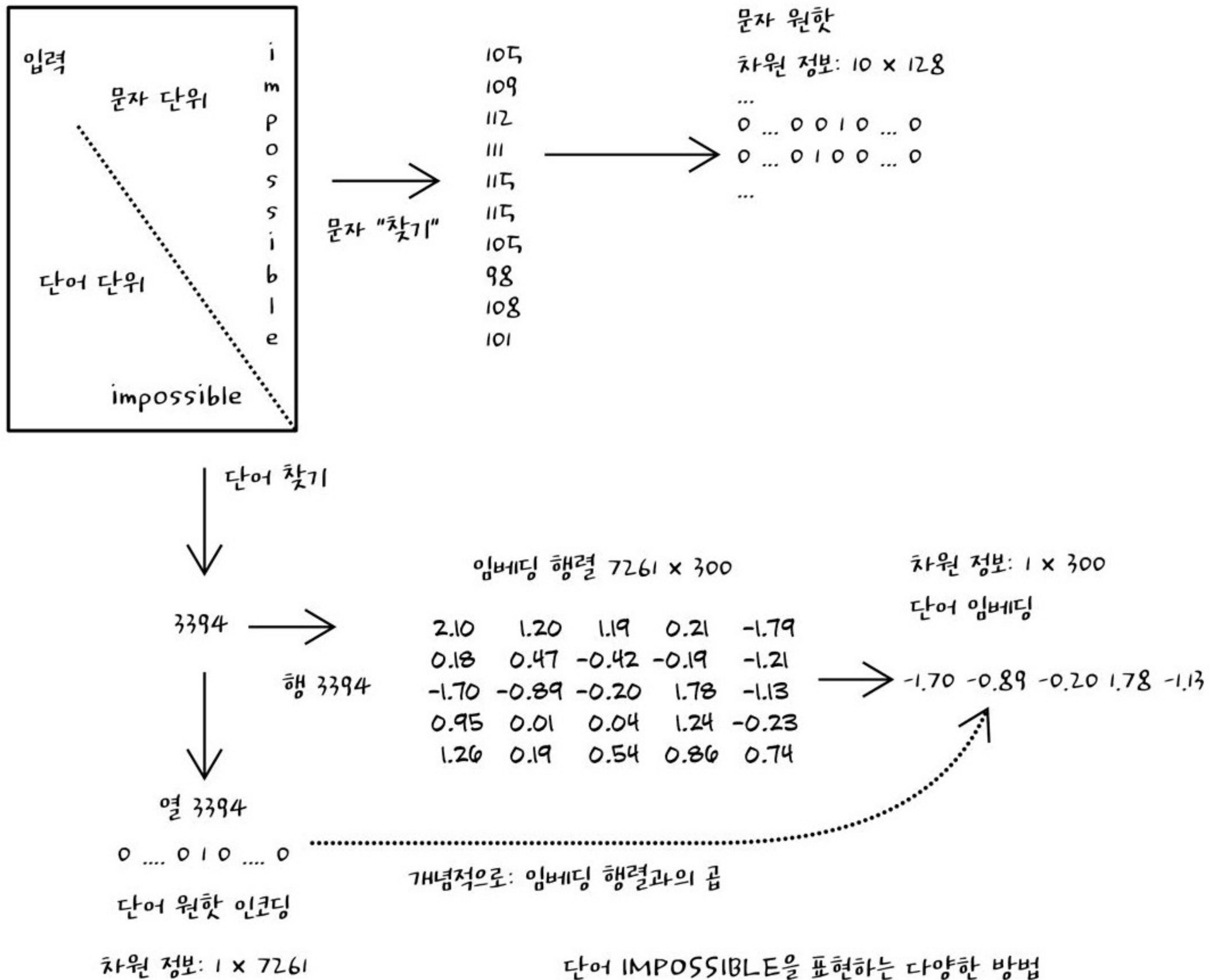
```
# Out[8]:
0 3394 impossible
1 4305 mr
2 813 bennet
3 3394 impossible
4 7078 when
5 3315 i
6 415 am
7 4436 not
8 239 acquainted
9 7148 with
10 3215 him
torch.Size([11, 7261])
```

이제 tensor는 사전에 들어 있는 단어 개수인 7,261의 인코딩 공간에서 길이가 11인 한 문장을 표현하게 됐다. 그림 4.6에서는 다음 절에서 보게 된 임베딩을 사용한 방식을 비롯해, 지금 살펴본 두 가지 인코딩 방식을 비교하고 있다.

문자 레벨과 단어 레벨 인코딩은 장단점이 있다. 대부분의 언어는 단어 수보다 문자 수가 훨씬 적다. 문자를 표현해 사용하면 표현할 수 있는 클래스도 몇 개 되지 않는다. 반면 단어 레벨은 매우 큰 수의 클래스를 표현할 수 있으며 실제 애플리케이션에서는 사전에 없는 단어도 다루게 된다. 단어는 개별 문자보다 더 많은 의미를 내포하므로 단어의 표현은 자체적으로 훨씬 많은 정보를 갖게 된다. 둘 간의 차이가 너무나 극명하기 때문에 이를 보완하는 방식이 발견되고 응용되어 큰 효과를 낸 사실은 그다지 놀랍지 않다. 예를 들어 **바이트 쌍 인코딩** byte pair encoding⁸ 사전에 개별 문자를 넣고 시작해서 지정된 사전 크기에 도달할 때까지 가장 많이 발견된 쌍을 되풀이하여 사전에 넣는 식으로 해결할 수 있다. 예제 문장에 이 방식을 사용하면 다음과 같은 토큰으로 쪼개어진다.⁹

```
?I|p|o|s|s|i|b|l|e|,|?M|r|.|?B|e|n|n|e|t|,|?i|m|p|o|s|s|i|b|l|e|,|?w|h|e|n|?I|?a|m|?n|o|t| ➡  
?a|c|q|u|a|i|n|t|e|d|?w|i|t|h|?h|i|m
```

대부분의 경우 매핑은 단어 단위로 구분하게 되지만, Impossible이나 이름인 Bennet같이 대문자로 시작하는 경우에는 세부 단위를 나누기도 한다.



4.5.4 텍스트 임베딩

원핫 인코딩은 텐서에서 카테고리 데이터를 표현할 때 매우 유용한 기술이다. 하지만 쉽게 예상할 수 있듯이 말뭉치의 단어에 해당하는 인코딩할 아이템 수를 효과적으로 제한하기 어려우면 실패하기 시작한다. 책 한 권에 이미 7,000개의 아이템이 들어 있다!

때문에 중복된 단어를 제거하고 철자를 대체해서 가짓수를 줄이거나, 과거 시제나 미래 시제를 구분하지 않고 하나의 토큰으로 보는 등의 방법이 필요하다. 범용 영어 인코딩의 크기는 매우 크다. 설상가상으로, 새로운 단어가 생기면 벡터에 열을 추가하여 모델이 새 사전 항목에 대한 가중치를 조정하도록 수정해야 하는데 이는 모델 훈련 관점에서 여간 성가신 문제가 아닐 수 없다.

어떻게 하면 인코딩 크기를 처리할 만한 규모로 줄이고 더 이상 늘어나지 않게 할 수 있을까? 하나만 값이 1이고 나머지는 0인 벡터 대신에, 부동소수점 수를 가지는 벡터를 사용해볼 수 있다. 100개의 부동소수점을 가진 벡터라면 엄청나게 많은 수의 단어를 표현할 수 있다. 이 방법의 핵심은 개별 단어를 100차원 공간에 매핑하여 이후 학습을 가능하게 하는 것이다. 이와 같은 방법을 임베딩(embedding)이라 부른다.

이론상으로는 사전을 단순히 순회하면서 각 단어마다 100개의 부동소수 난수로 구성된 집합을 생성할 수 있다. 엄청나게 많은 단어라고 해도 손쉽게 숫자 100개 안에 다 들어간다. 하지만 이렇게 해버리면 의미나 문맥에 기반해 단어 사이의 거리를 두도록 배치하는 개념을 포기하는 것과 마찬가지다. 이런 식의 단어 임베딩 모델을 사용한다면 입력 벡터의 구조로부터 다룰 만한 내용이 거의 없게 된다. 이상적인 방법이라면, 비슷한 맥락에서 사용한 단어들이 가까운 거리에 배치되는 임베딩을 만들어 사용하는 것이다.

여기서 이 문제를 수동으로 푸는 방안을 설계한다면, 아마 기본 체언(명사)이나 용언(형용사)을 차원의 축을 따라 매핑하며 임베딩 공간을 구축하게 될 것이다. 2차원 공간을 생성할 때 각 축은 과일(0.0-0.33), 꽃(0.33-0.66), 강아지(0.66-1.0)처럼 체언에 매핑하며, 붉은(0.0-0.2), 주황인(0.2-0.4), 노란(0.4-0.6), 하얀(0.6-0.8), 갈색인(0.8-1.0)처럼 용언을 매핑하게 된다. 이와 같은 작업은 실제 과일과 꽃, 강아지라는 단어를 임베딩에 배치하기 위함이다.

이제 단어 임베딩을 시작하면, 우리는 ‘사과’를 ‘과일’과 ‘붉은’이라는 단어의 사분면에 매핑할 수 있다. 같은 식으로 ‘귤’, ‘레몬’, ‘리치’, ‘키위’ 등의 과일도 색상에 맞춰 쉽게 매핑할 수 있다. 이어서 꽃으로 넘어가서 ‘장미’, ‘양귀비’, ‘수선화’, ‘백합’ 등도 하면 되는데... 음, 갈색인 꽃은 별로 없는 것 같다. ‘해바라기’는 ‘꽃’과 ‘노란’, ‘갈색인’을 얻을 수 있고, ‘데이지’는 ‘꽃’과 ‘하얀’, ‘노란’을 얻는다. ‘키위’는 ‘과일’과 ‘갈색인’, ‘녹색인’에 가깝게 매핑해야 할 것 같다.¹⁰ 이제 강아지와 색으로 넘어가서, ‘레드본’ 종의 강아지는 ‘붉은’ 근처에, ‘여우’는 ‘주황인’에 놓고, ‘골든 리트리버’는 ‘노란’으로, ‘푸들’은 ‘하얀’... 그리고 대부분의 견종은 ‘갈색인’으로 임베딩이 가능하다.

이제 임베딩이 그림 4.7과 비슷해졌다. 하지만 이렇게 임베딩을 수동으로 배치하는 작업은 큰 말뭉치의 경우에는 현실적이지 않다. 2차원의 임베딩에 대해 8개의 기본 단어로 15개의 다른 단어 배치를 설명했는데 시간을 들여서 창의력을 발휘해봐야 겨우 몇 개 더 넣을 수 있을 듯하다.

짐작했을지 모르겠지만, 이 작업은 자동화할 수 있다. 원문 텍스트로 구성된 다량의 말뭉치를 잘 처리하면 방금 설명한 것과 유사한 임베딩을 구축할 수 있다. 차이점을 살펴보면, 임베딩 벡터에는 100에서 1000개 구성 요소가 존재하며, 축은 기반 개념에 직접 매핑되지 않는다. 그 대신, 개념적으로 유사한 단어들은 임베딩 공간의 인접 영역에 매핑되며, 임베딩 공간의 축은 임의의 부동소수점 차원이 된다.

방금 설명한 구체적인 알고리즘¹¹은 이 책의 내용에서 벗어나니 신경망을 사용해서 임베딩이 만들어질 수 있다는 것과 이를 통해 문장에서 개념상으로 가까운 단어를 예측해낼 수 있다는 정도만 알아두자. 지금의 경우는 원핫 인코딩된 단어를 (다소 얇은) 신경망을 사용해 임베딩을 만들 수 있다. 이렇게 만든 임베딩을 후반 작업에서 사용하게 된다.

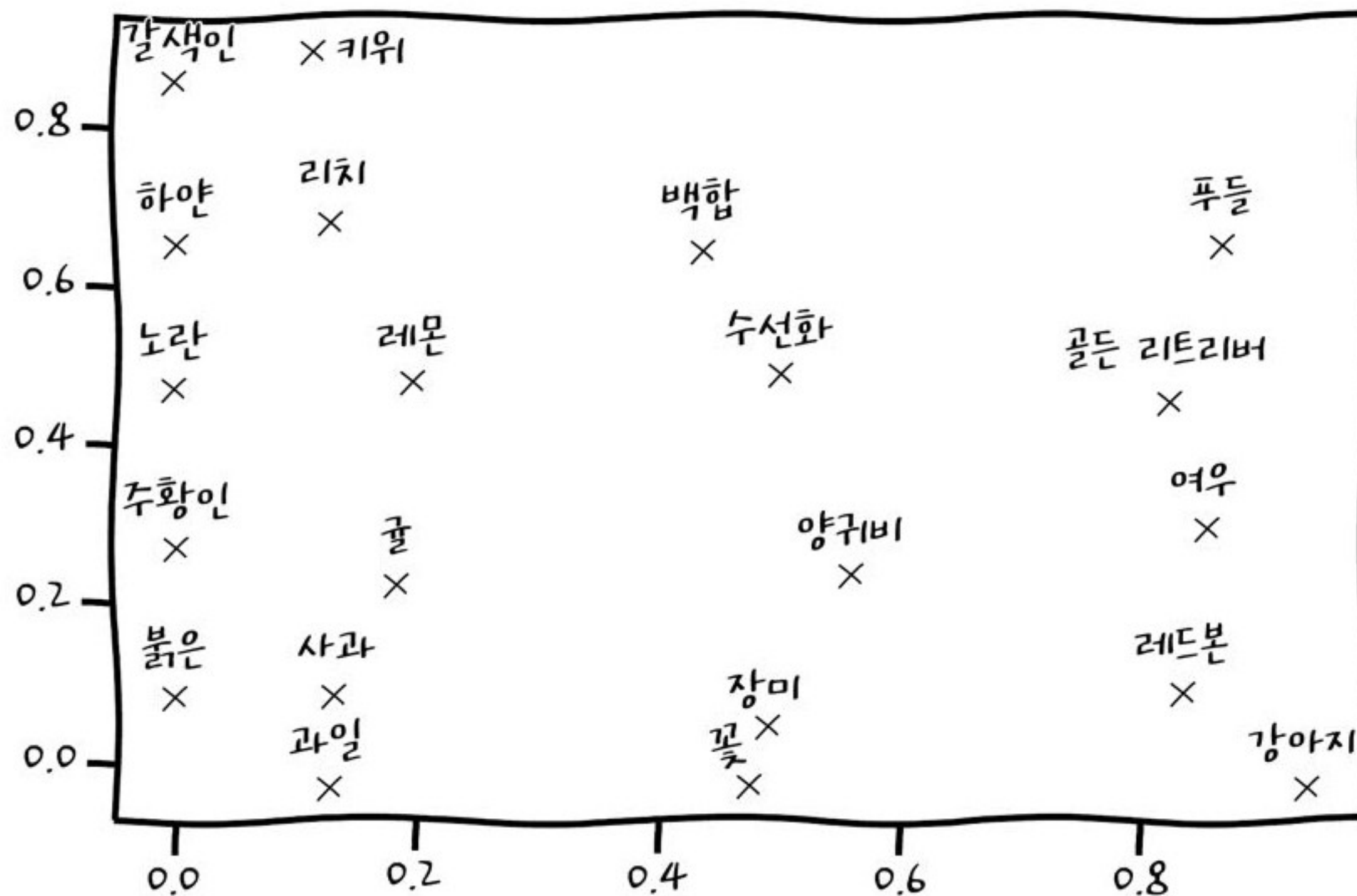


그림 4.7 수작업으로 만든 단어 임베딩

한 가지 흥미로운 점은 만들어진 임베딩에서 비슷한 단어들끼리 군집할 뿐 아니라 일관된 공간 관계를 유지한다는 점이다. 예를 들어 사과라는 단어의 임베딩 벡터를 찾아 다른 단어를 더하거나 빼서 비유 관계를 잡아낼 수 있는데, ‘사과 - 붉은 - 달콤한 + 노란 + 새콤한’처럼 계산하면 결과 벡터는 레몬 벡터와 상당히 유사해진다.

대중 매체의 헤드라인을 장식하는 BERT나 GPT-2를 사용한 현대식 임베딩 모델은 더 정교할 뿐 아니라 문맥에 민감한 모습을 보이는데, 사전의 단어 매핑이 특정 위치로 항상 고정되지 않고 주변을 둘러싼 문장에서 영향을 받는다. 그리고 지금 우리가 다루는 전형적인 방식의 임베딩처럼 사용할 수 있다.

4.5.5 청사진으로서의 텍스트 임베딩

어휘 집합 내의 많은 개체가 숫자 벡터로 표현되어야 할 경우 임베딩은 필수적이다. 하지만 이 책에서는 텍스트나 텍스트 임베딩을 사용하지 않을 예정이다. 사용하지 않을 건데 왜 소개했는지 궁금할지도 모르겠다. 우리는 텍스트를 표현하고 처리하는 방법을 일반적으로 카테고리 데이터를 다루는 예제로도 볼 수 있다고 믿는다. 원핫 인코딩이 부담스러운 경우라면 임베딩이 유용하다. 앞에서 다룬 형태가 임베딩 벡터를 행으로 가지는 행렬의 곱셈 연산을 수행하기 위한 효율적인 원핫 인코딩 표현 방법으로 보면 된다.

텍스트가 아닌 경우 일반적으로 사전에 임베딩을 만들 수는 없지만 앞에서는 다루지 않았던 임의의 숫자로 시작해서 우리의 학습 문제의 일부로 놓고 발전시켜 가는 법을 고려할 수 있다. 이것은 표준적인 방법이기도 한데 이 때문에 임베딩은 모든 카테고리 데이터에 대한 원핫 인코딩의 훌륭한 대체재가 된다. 다른 한편으로, 텍스트를 다루는 문제를 풀 때 사전에 학습된 임베딩을 개선시켜 가는 것도 일반적인 방법이다.¹²

만일 동시에 발견되는 부분에 관심이 있다면, 앞에서 살펴본 단어 임베딩은 일종의 청사진 역할을 한다. 예를 들어 “이 책을 좋아하는 고객이 ...를 구입했어요.” 같은 추천 시스템은 고객이 이미 상호작용한 다른 아이템을 다양 한 관심사를 예측할 때의 맥락으로 사용한다. 마찬가지로, 텍스트 처리는 연속된 데이터를 다루는 작업 중 아마도 가장 흔하고 많이 연구된 것 중 하나일 것이다. 그래서 시계열 데이터를 다루는 작업을 하다가 자연어 처리 분야의 힌트를 얻게 되기도 한다.

4.6 결론

이 장에서는 많은 기반을 다졌다. 가장 흔한 형태의 데이터를 어떻게 읽어들이는지와 신경망에 넣기 위한 변환에 대해 배웠다. 물론 실제로는 하나의 덩어리로 기술하기를 희망하는 경우보다 현실에서는 더 다양한 데이터 포맷이 존재한다. 예를 들어 의료 기록 같은 경우는 너무 복잡해서 여기에서 다루기는 어렵다. 오디오나 비디오 같은 형태는 이 책에서 꼭 다뤄야 할 형식은 아닌 것으로 판단했다. 그래도 관심 있다면 책의 웹사이트(www.manning.com/books/deep-learning-with-pytorch) 다운로드 링크에서 주피터 노트북을 제공하며, 코드 저장소(<https://github.com/deep-learning-with-pytorch/dlwpt-code/tree/master/p1ch4>)에서 오디오와 비디오 텐서 생성에 대한 짧은 예제를 보너스로 제공하니 참고하기 바란다.

이제는 텐서에 익숙해졌고 데이터를 텐서에 어떻게 저장하는지도 알았다. 이 책의 목표를 향해 다음 단계로 넘어가자. 심층 신경망 훈련 말이다! 다음 5장에서는 간단한 선형 모델의 학습 방법을 설명한다.

4.8 핵심 요약

- 신경망에서는 데이터가 다차원 숫자 텐서로 표현되어야 하며 대부분 32비트 부동소수점을 사용한다.
- 통상 파이토치에서 데이터는 컨볼루션이나 순환 등의 모델 아키텍처에 맞는 특정 차원으로 만들어져야 한다. 데이터는 파이토치 텐서 API를 사용하면 효율적으로 변환할 수 있다.
- 감사하게도 파이토치 라이브러리는 파이썬 표준 라이브러리 및 관련 생태계와 긴밀하게 상호작용하므로 일반적인 데이터 타입을 읽어들이거나 파이토치 텐서로 변환하기가 매우 편리하다.
- 이미지는 한 개 이상의 채널로 구성되어 있다. 대부분은 디지털 사진의 적-녹-청(RGB) 채널에 해당한다.
- 많은 경우 이미지는 채널당 8비트를 가지며 12나 16비트도 특이한 경우는 아니다. 이런 비트 깊이는 모두 정밀도 손실 없이 32비트 부동소수점 수로 보관할 수 있다.
- 채널이 하나인 데이터는 채널 자체의 차원을 생략한 경우가 있다.
- 용적 데이터는 2차원 이미지 데이터와 유사하며 심도 같은 세 번째 차원이 존재하는 차이가 있다.
- 스프레드시트를 텐서로 변환하는 것은 매우 단순하다. 이때 카테고리값이나 순서값이 들어있는 열은 이산 값으로 이뤄진 열과는 다른 방식으로 처리해야 한다.
- 텍스트나 카테고리 데이터는 사전을 사용한 원핫 표현으로 인코딩할 수 있다. 많은 경우 임베딩은 효율적이면서 좋은 표현 방법이다.